

Artificial Neural Network-Based Exchange and Correlations Functionals

João Paulo Almeida de Mendonça (*he/him*)

CNRS, SIMaP, Grenoble, France

joao-paulo.almeida-de-mendonca@grenoble-inp.fr

 **anr**[®]
agence nationale
de la recherche

What we will cover on this talk?

- Density Functional Theory (DFT)
 - What is DFT? What can we do with it?
 - Theoretical Background
 - How to expand it?
- Artificial Neural Network (ANN)
 - What are ANNs? Why they are interesting do DFT?
 - Understanding Feedforward Neural Networks
 - How can it be a DFT Functional?
 - Challenges in Training
- ANN Functionals
 - Some examples on the literature and how they do it
 - My work and how we do it at our group
- Preparing for the practical section!

Please, help me by making questions!

This is a very interdisciplinary topic
and no one is expected to know
everything about it.

If not now, stop me at some moment
on the conference and ask!

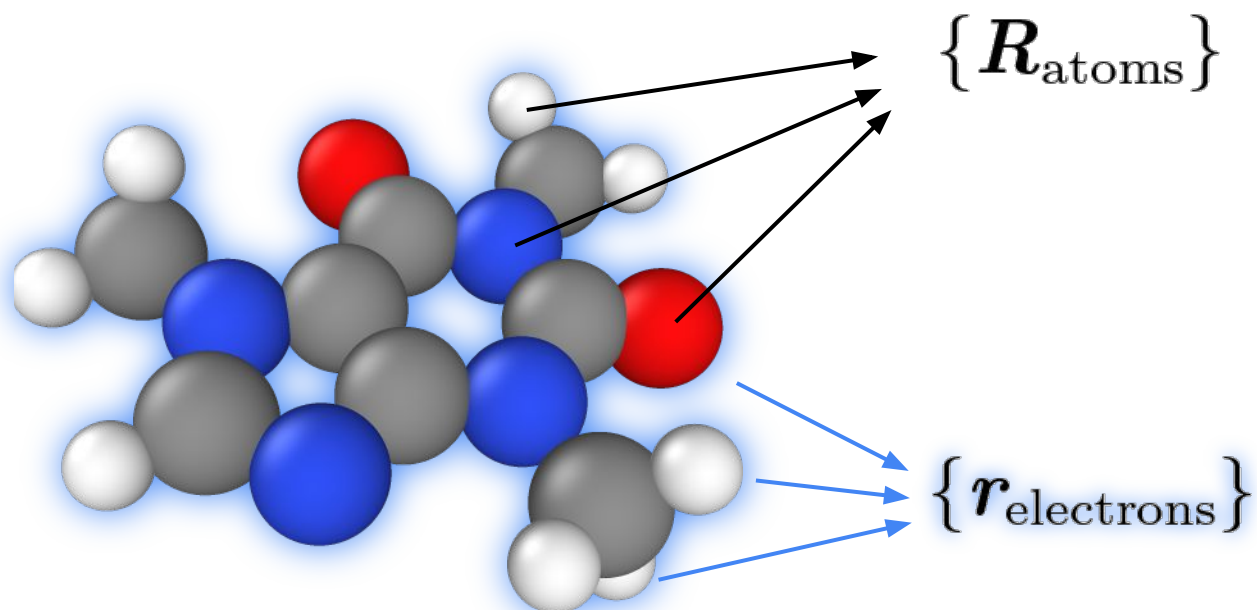
*"We all know something. We all choose
to ignore something. And that's why we
always learn." (Paulo Freire)*

What is Density Functional Theory (DFT)?

“A theory used to describe **many-fermion systems** in which **the energy is a functional of the density of fermions**. Density functional theory has been used extensively in the theory of **electrons in atoms, molecules, and solids** and in the theory of nucleons in nuclei.”

(From A Dictionary of Physics - Oxford University Press)

How this is done?



$$H_{\text{elec.}+\text{atoms}} = \underbrace{-\sum_i \frac{1}{2} \nabla_i^2 - \sum_A \frac{1}{2M_A} \nabla_A^2}_{\text{Kinetic Energy}} - \underbrace{\sum_{i,A} \frac{Z_A}{r_{iA}} + \sum_{i>j} \frac{1}{r_{ij}} + \sum_{B>A} \frac{Z_A Z_B}{R_{AB}}}_{\text{Potential Energy}}$$

$$H_{\text{elec.}+\text{atoms}} = - \sum_i \frac{1}{2} \nabla_i^2 - \sum_A \frac{1}{2M_A} \nabla_A^2 - \sum_{i,A} \frac{Z_A}{r_{iA}} + \sum_{i>j} \frac{1}{r_{ij}} + \sum_{B>A} \frac{Z_A Z_B}{R_{AB}}$$

a proton is ~1840 times the electron mass

Born-Oppenheimer Approximation:

- The use an approximate ansatz for the complete many-body problem that **allows the wave functions of atomic nuclei and electrons to be treated separately.**
- This approximation is motivated by the mass difference between nuclei and electrons.
- The electronic Hamiltonian than can be written as the **kinetic energy of the electrons, the electron interaction with fixed nuclei, and the electron-electron interaction.**

$$\hat{\mathbf{H}} = - \sum_i \frac{1}{2} \nabla_i^2 - \sum_{i,A} \frac{Z_A}{r_{iA}} + \sum_{i>j} \frac{1}{r_{ij}}$$

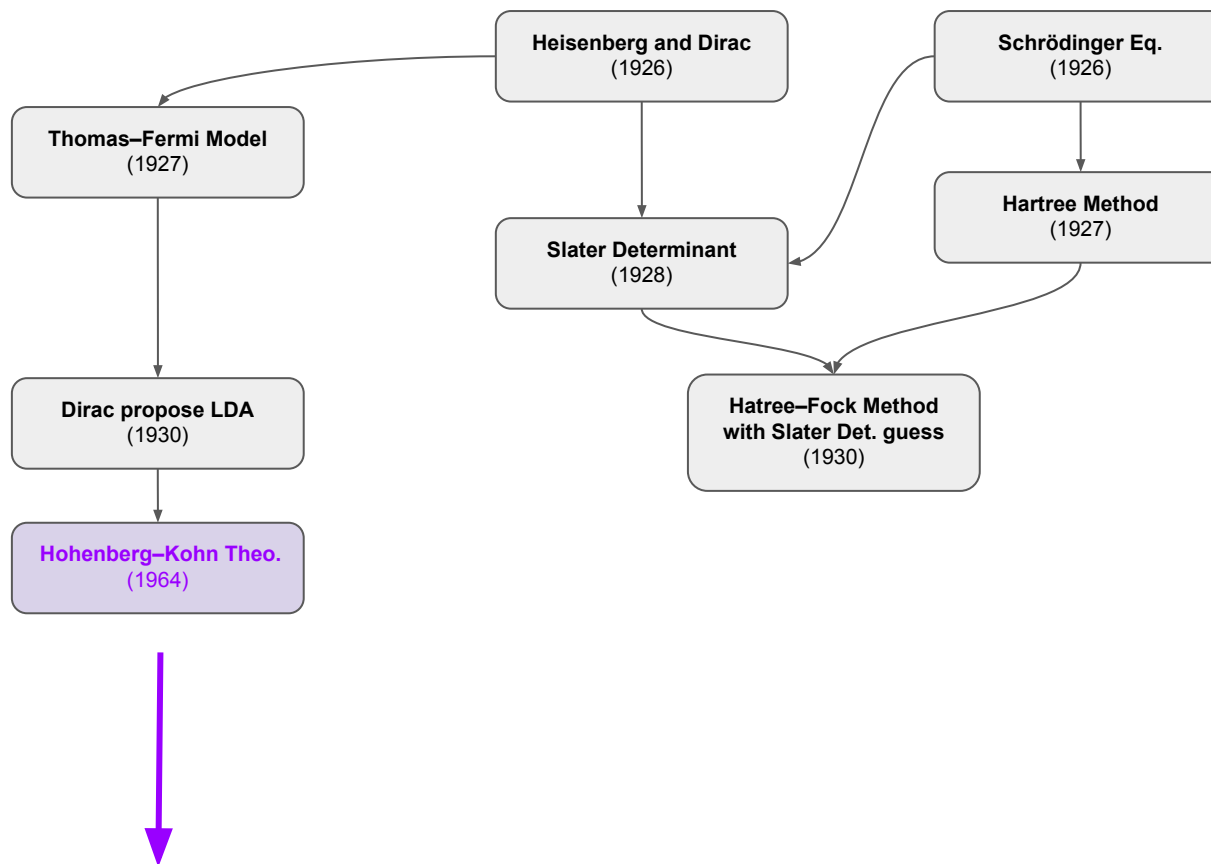
$$\left[-\sum_i^N \frac{1}{2} \nabla_i^2 - \sum_{i,A}^{N, N_{\text{atoms}}} \frac{Z_A}{r_{iA}} + \sum_{i>j}^{N,N} \frac{1}{r_{ij}} \right] \Psi(\mathbf{r}_1, \dots, \mathbf{r}_N) = E \Psi(\mathbf{r}_1, \dots, \mathbf{r}_N)$$

The normalized electronic wavefunction Ψ can
be associated to a electronic density $n(\mathbf{r})$:

$$n(\mathbf{r}) = N \int d^3\mathbf{r}_2 \cdots \int d^3\mathbf{r}_N \Psi^*(\mathbf{r}_1, \dots, \mathbf{r}_N) \Psi(\mathbf{r}_1, \dots, \mathbf{r}_N)$$

This definition **can be inverted for the ground state**, so the density $n_0(\mathbf{r})$ unambiguously defines $\Psi_0[n_0(\mathbf{r})]$, and as consequence defines the ground state energy of the electronic system $E_0[n_0(\mathbf{r})]$.

Hence, ***Density Functional Theory!***



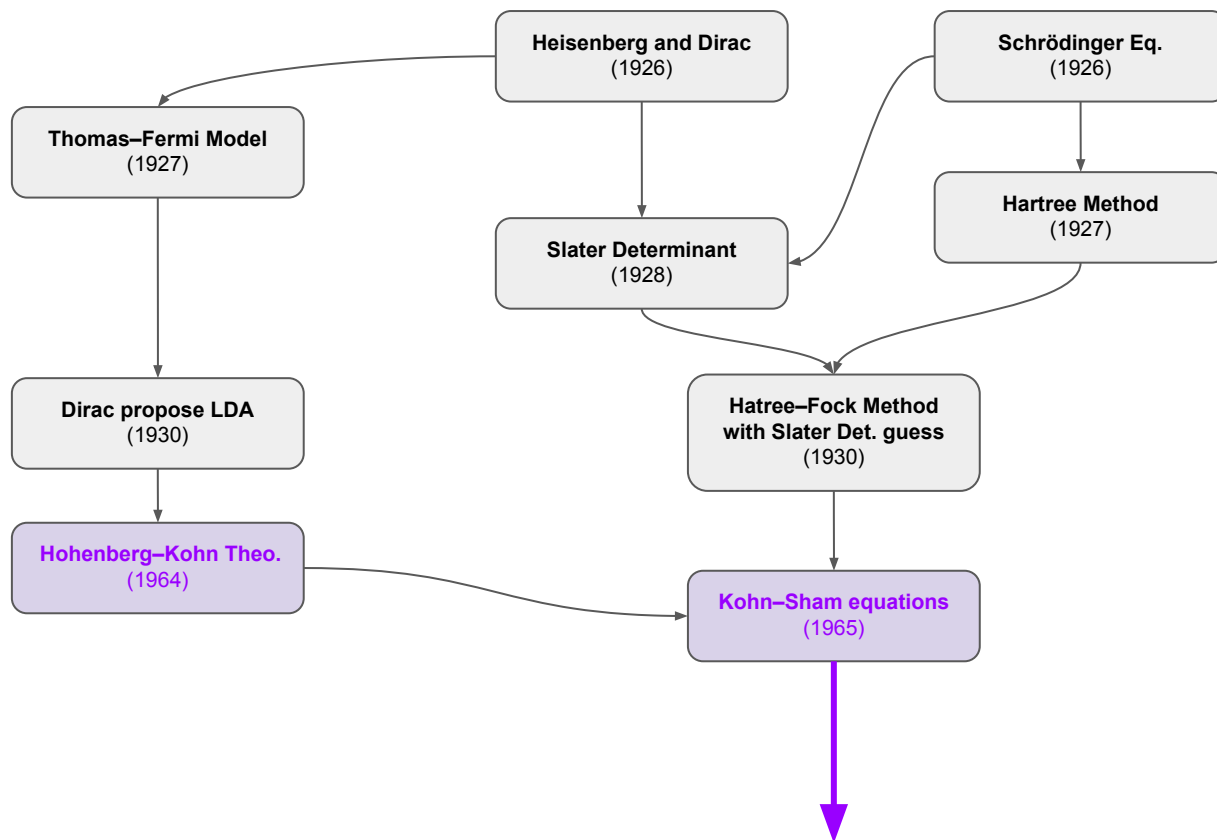
Theorem 1. The external potential v_{ext} (and hence the total energy E_{tot}), is a **unique functional of the electron density $n(\mathbf{r})$** .

Theorem 2. The functional that delivers the ground-state energy of the system ($E_{\text{tot}}[n(\mathbf{r})]$) **gives the lowest energy E_0 if and only if the input density is the true ground-state density.**



VERY GENERAL

Valid for all systems where $v_{\text{ext}}(\mathbf{r})$ defines a pure ground state!



Single non-interacting particle approximation for $n(\mathbf{r})$:

$$n(\mathbf{r}) = \sum_i^N |\varphi_i(\mathbf{r})|^2$$

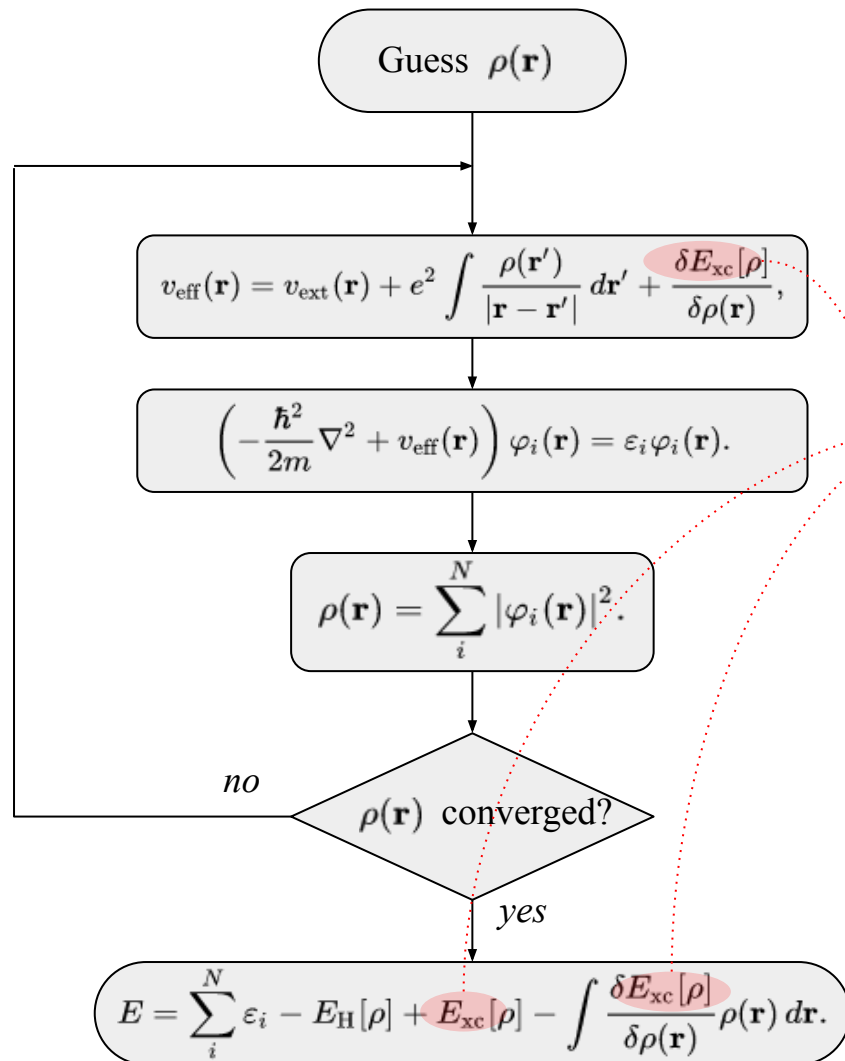
The single particle eigenstate problem can be solved using the **Kohn–Sham equations**:

$$\left[-\frac{\hbar^2}{2m} \nabla^2 + V_{\text{ext}}(\mathbf{r}) + \int \frac{n(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d^3\mathbf{r}' + V_{\text{xc}}[n(\mathbf{r})] \right] \varphi_i(\mathbf{r}) = \varepsilon_i \varphi_i(\mathbf{r})$$

We can recover the energy in the many-electrons system:

$$E = \sum \varepsilon_i - \frac{1}{2} \iint \frac{n(\mathbf{r})n(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d\mathbf{r}' d\mathbf{r} + E_{\text{xc}}[n(\mathbf{r})] - \int \frac{\delta E_{\text{xc}}[n(\mathbf{r})]}{\delta n(\mathbf{r})} n(\mathbf{r}) d\mathbf{r}.$$

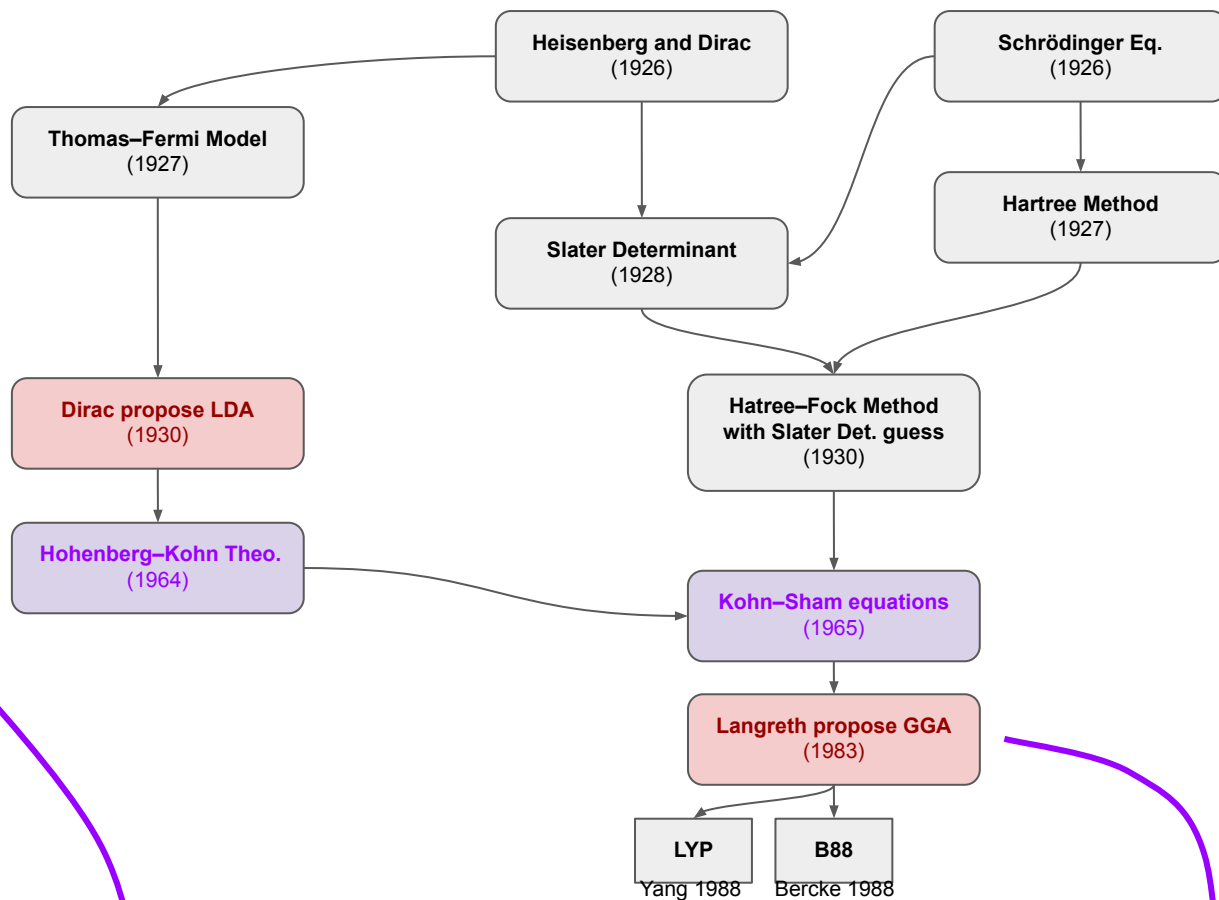
Kohn–Sham equation solved self-consistently



$$E_{\text{xc}}[\rho(\mathbf{r})] = \int \rho(\mathbf{r}) \varepsilon_{\text{xc}}[\rho(\mathbf{r}), \nabla \rho(\mathbf{r}) \dots] d\mathbf{r}$$

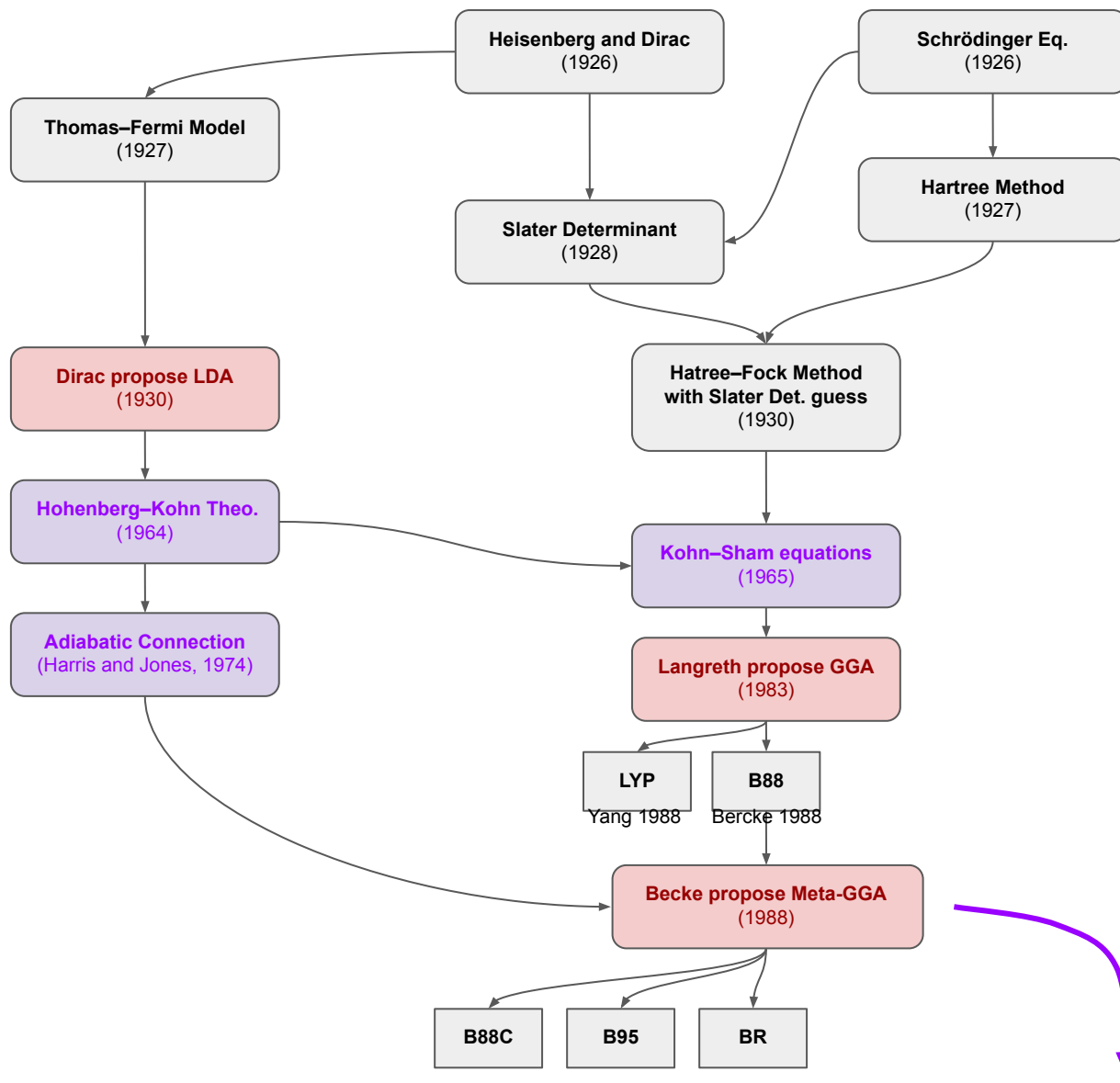
We know from the Hohenberg–Kohn Theorems that this ε_{xc} exist, but unfortunately it can not be derived from first principles.

Building good approximations for this functional is on the core of the quality of DFT results and applicability.



$$E_{xc}^{LDA}[n] = \int n(\mathbf{r}) \varepsilon_{xc}[n] d\mathbf{r}$$

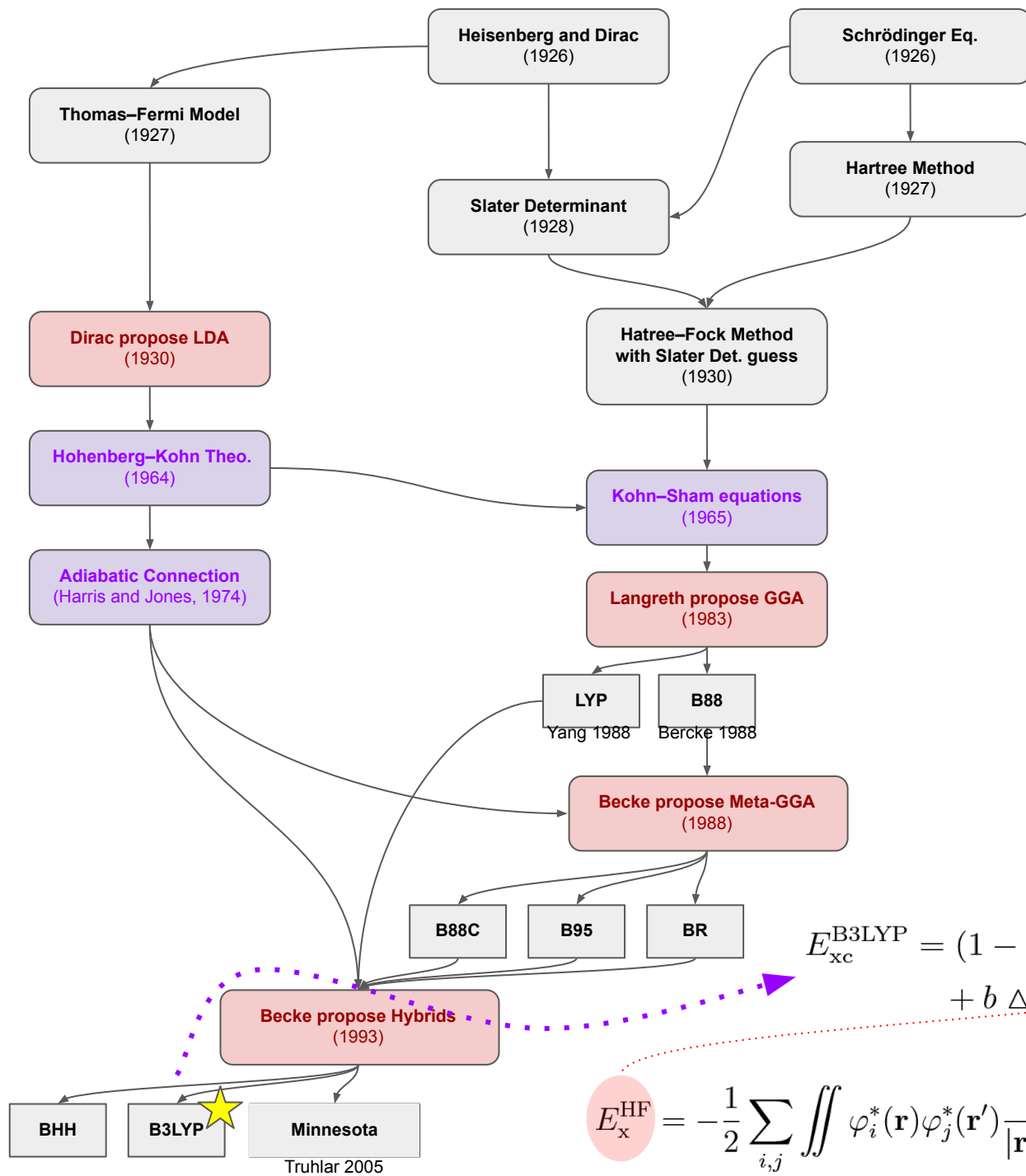
$$E_{xc}^{GGA}[n] = \int n(\mathbf{r}) \varepsilon_{xc}[n, \nabla n,] d\mathbf{r}$$



Kinetic-energy density

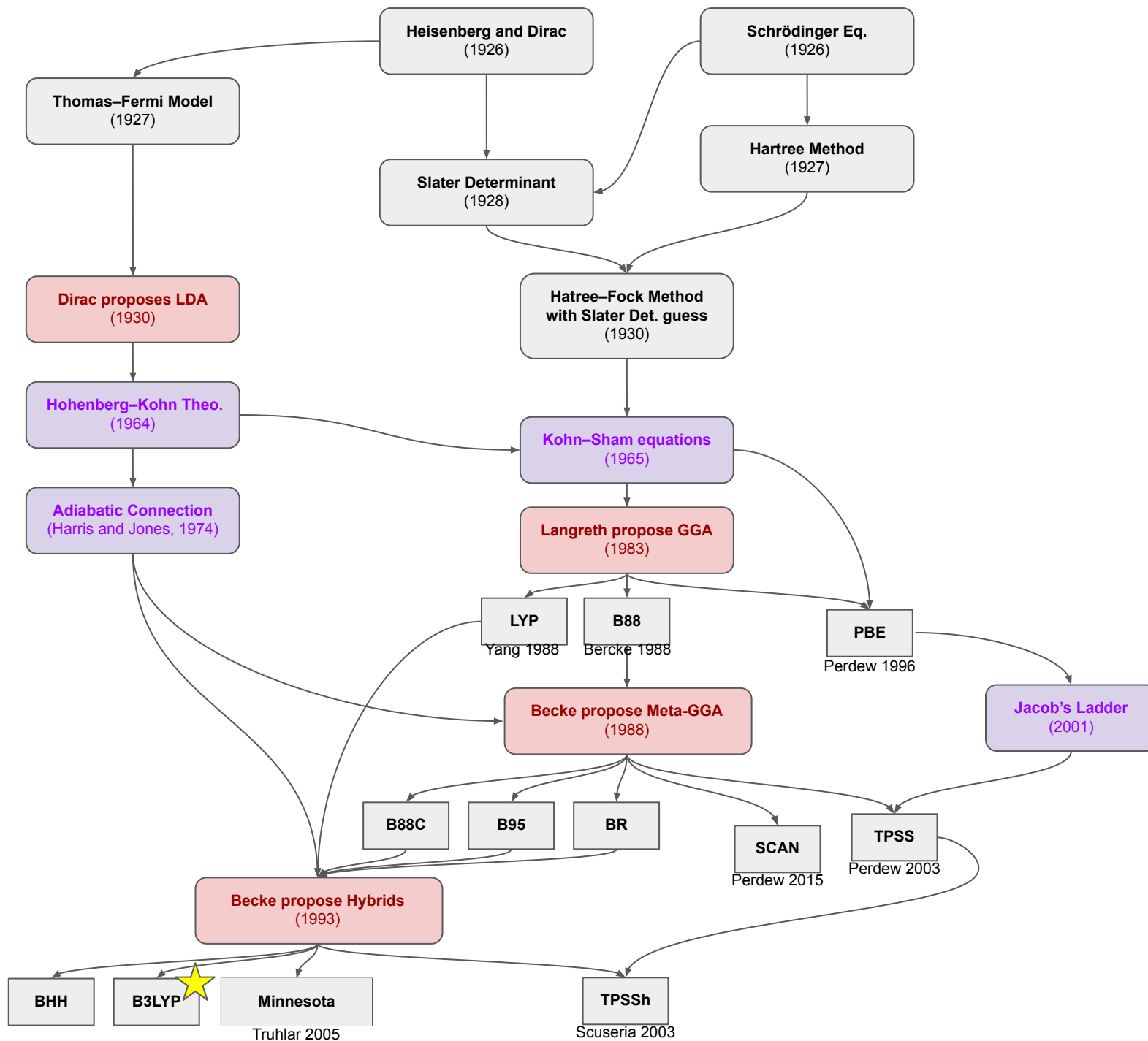
$$\tau = \frac{1}{2} \sum_i^{\text{occ}} |\nabla \varphi_i|^2$$

$$E_{\text{xc}}^{\text{meta-GGA}}[n] = \int n(\mathbf{r}) \varepsilon_{\text{xc}}[n, \nabla n, \tau] d\mathbf{r}$$



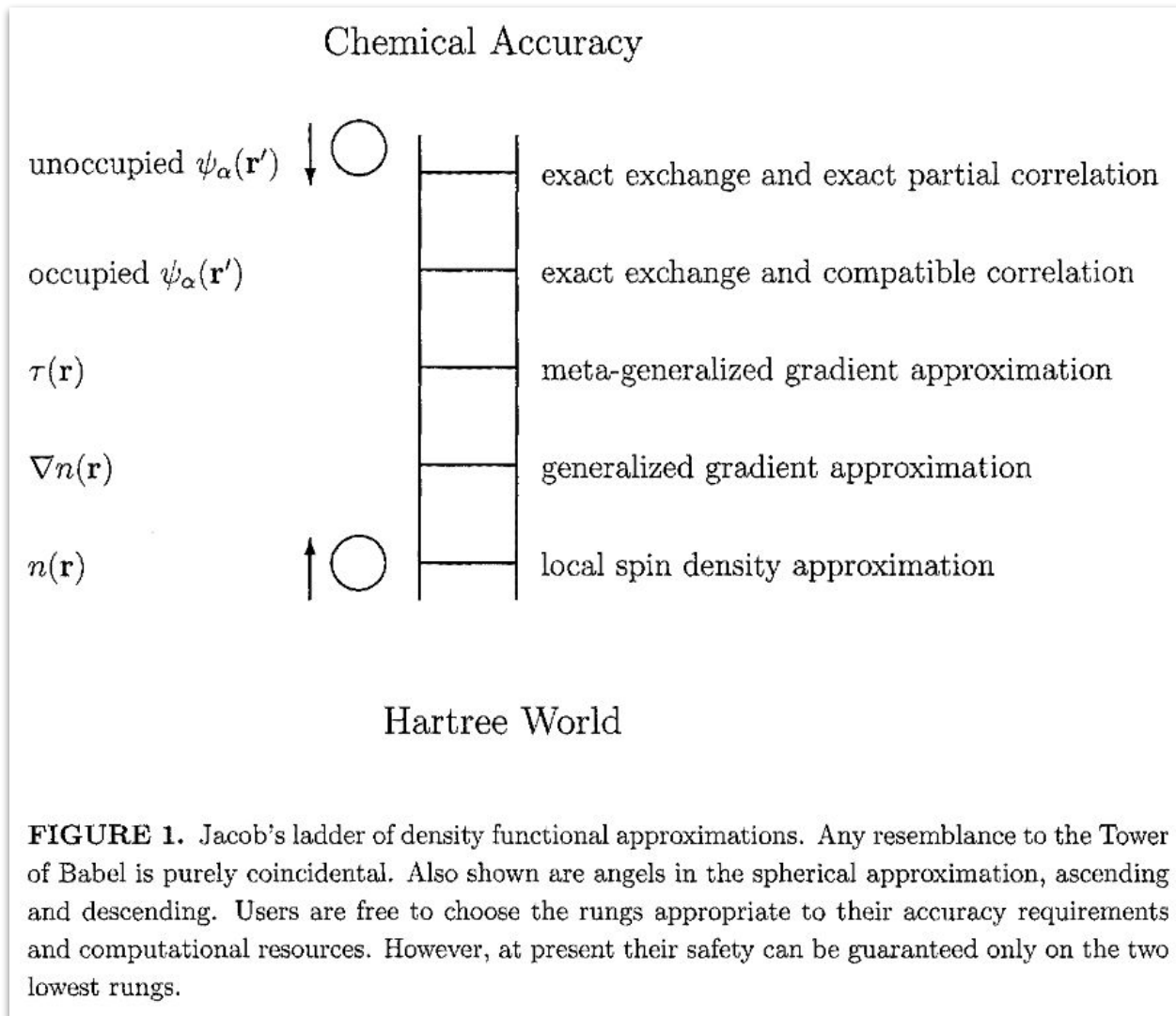
$$E_{xc}^{B3LYP} = (1 - a)E_x^{LDA} + aE_x^{HF} + b \Delta E_x^{B88} + (1 - c)E_c^{LDA} + cE_c^{LYP}$$

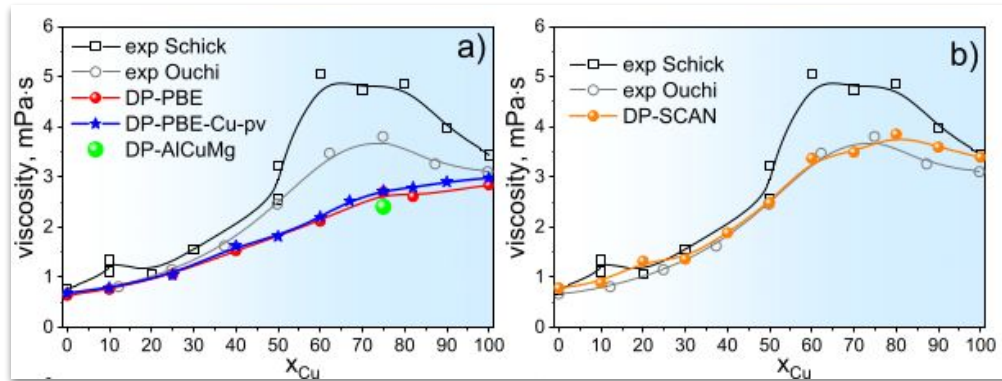
$$E_x^{HF} = -\frac{1}{2} \sum_{i,j} \iint \varphi_i^*(\mathbf{r}) \varphi_j^*(\mathbf{r}') \frac{1}{|\mathbf{r} - \mathbf{r}'|} \varphi_j(\mathbf{r}) \varphi_i(\mathbf{r}') d\mathbf{r} d\mathbf{r}'$$



“And he[Jabob] dreamed, and behold a ladder set up on the earth, and the top of it reached to heaven; and behold the angels of God ascending and descending on it.”

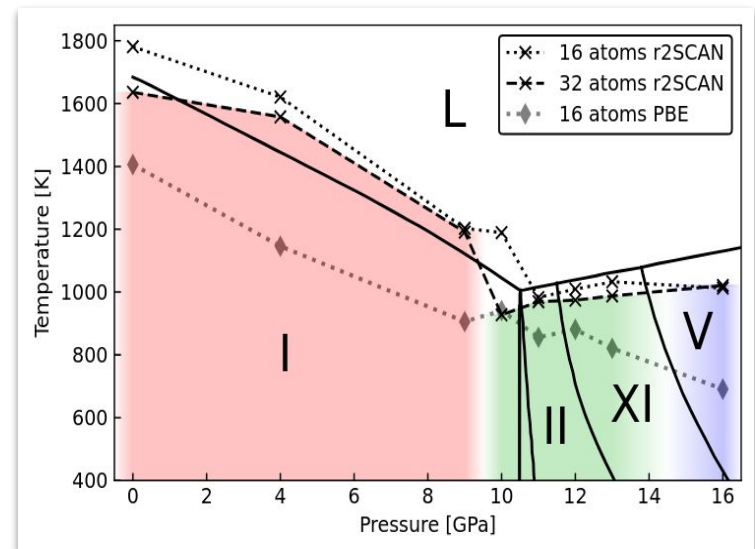
[Gen 28:12]





Al-Cu Melts

Khazieva et al. 2024 DOI: 10.1063/5.0222355

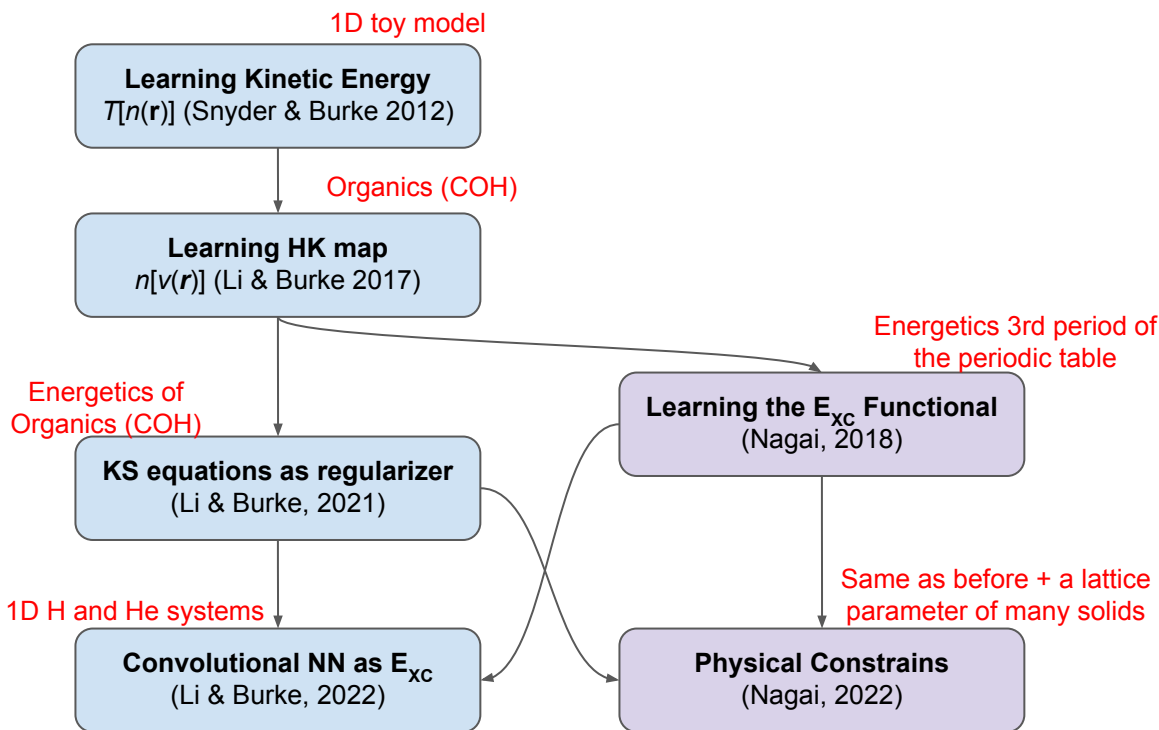


Si phase diagram

Unglert et al. 2023, arXiv:2308.11426

And where does Machine Learning appear?

- We are in the *Objective Oriented Fitting*: All work on machine learning functionals is on that category. Machine learning is a data driven approach, so its subject dependent by definition.



“... [ML approximations] achieves chemical accuracy using many more inputs, but requires far less insight into the underlying physics.”

Why Neural Networks and not a simpler thing?

- We want to approximate $E_{xc}[n(\mathbf{r})]$, that we know **exist and is unique**.
- But we also know that **the one-to-one correspondence between n and E_{xc} contains non-analytic structures** (e.g., discontinuities and singularities).
- Neural Networks are shown to be good **universal approximators** even in the case of intricate functions.

Multilayer Feedforward Networks are Universal Approximators

KUR' HORNIK

Technische Universität Wien

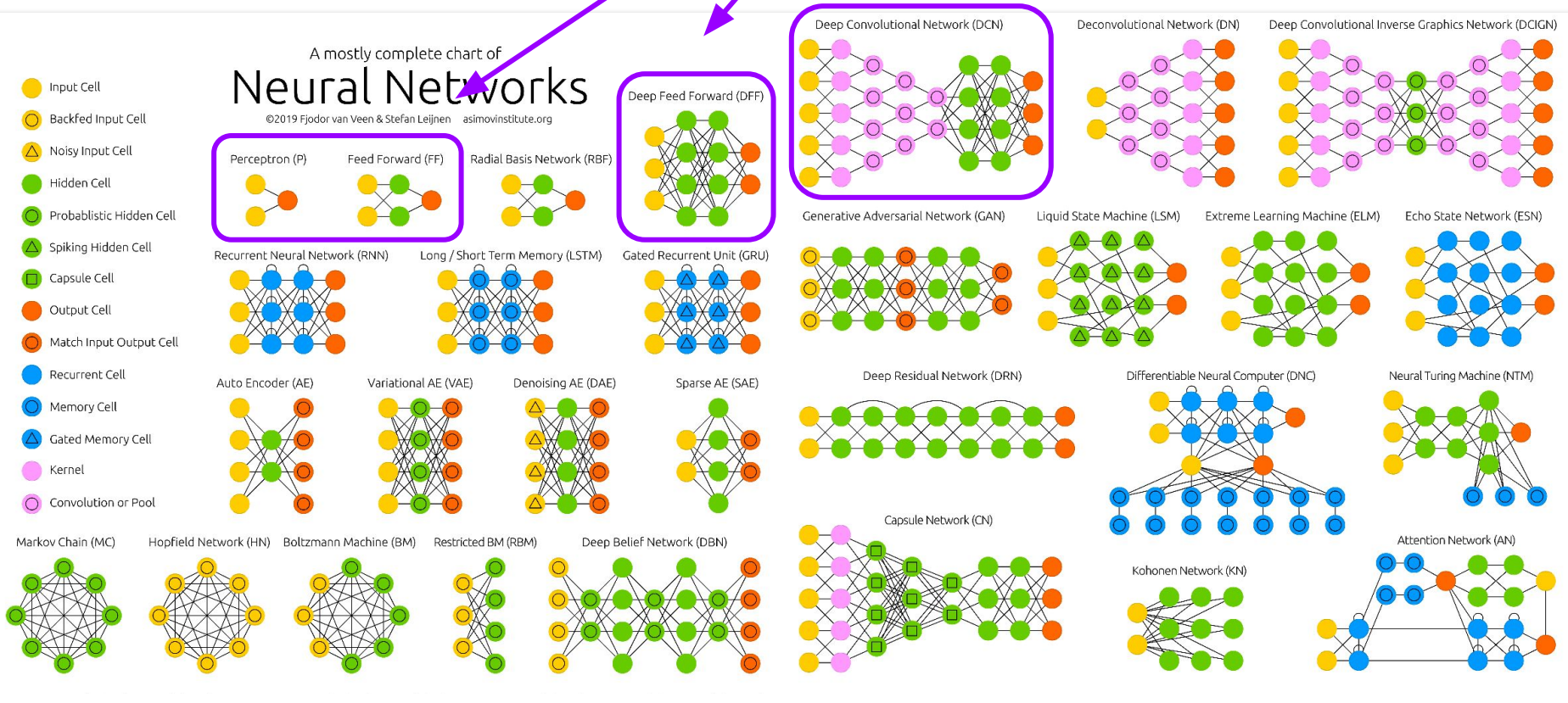
MAXWELL STINCHCOMBE AND HALBER WHITE

University of California, San Diego

(Received 16 September 1988; revised and accepted 9 March 1989)

Abstract—This paper rigorously establishes that standard multilayer feedforward networks with as few as one hidden layer using arbitrary squashing functions are capable of approximating any Borel measurable function from one finite dimensional space to another to any desired degree of accuracy, provided sufficiently many hidden units are available. In this sense, *multilayer feedforward networks are a class of universal approximators*.

Already used on Machine Learning Functionals



Already used on Machine Learning Functionals



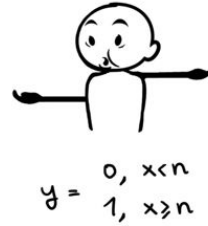
Sigmoid



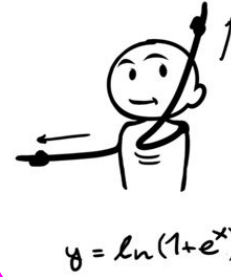
Tanh



Step Function



Softplus



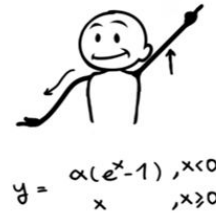
ReLU



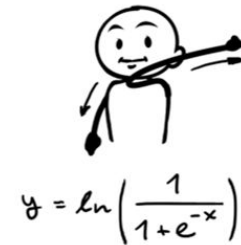
Softsign



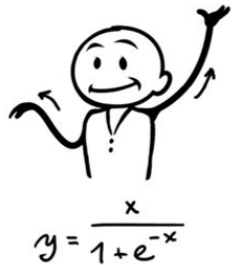
ELU



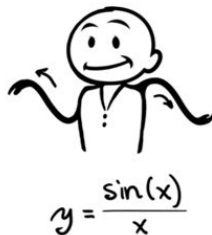
Log of Sigmoid



Swish



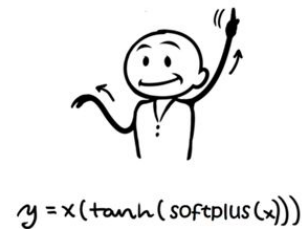
Sinc



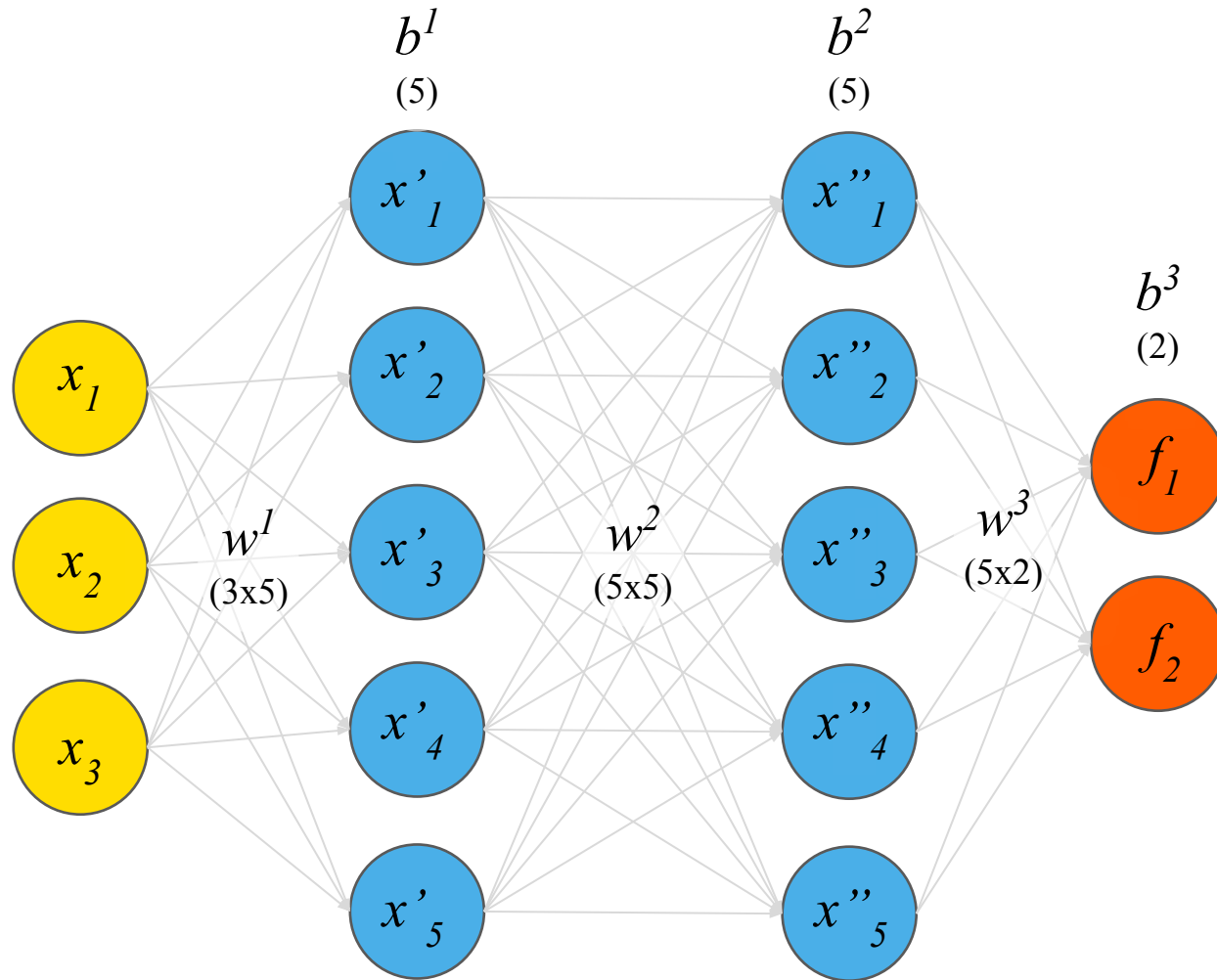
Leaky ReLU



Mish



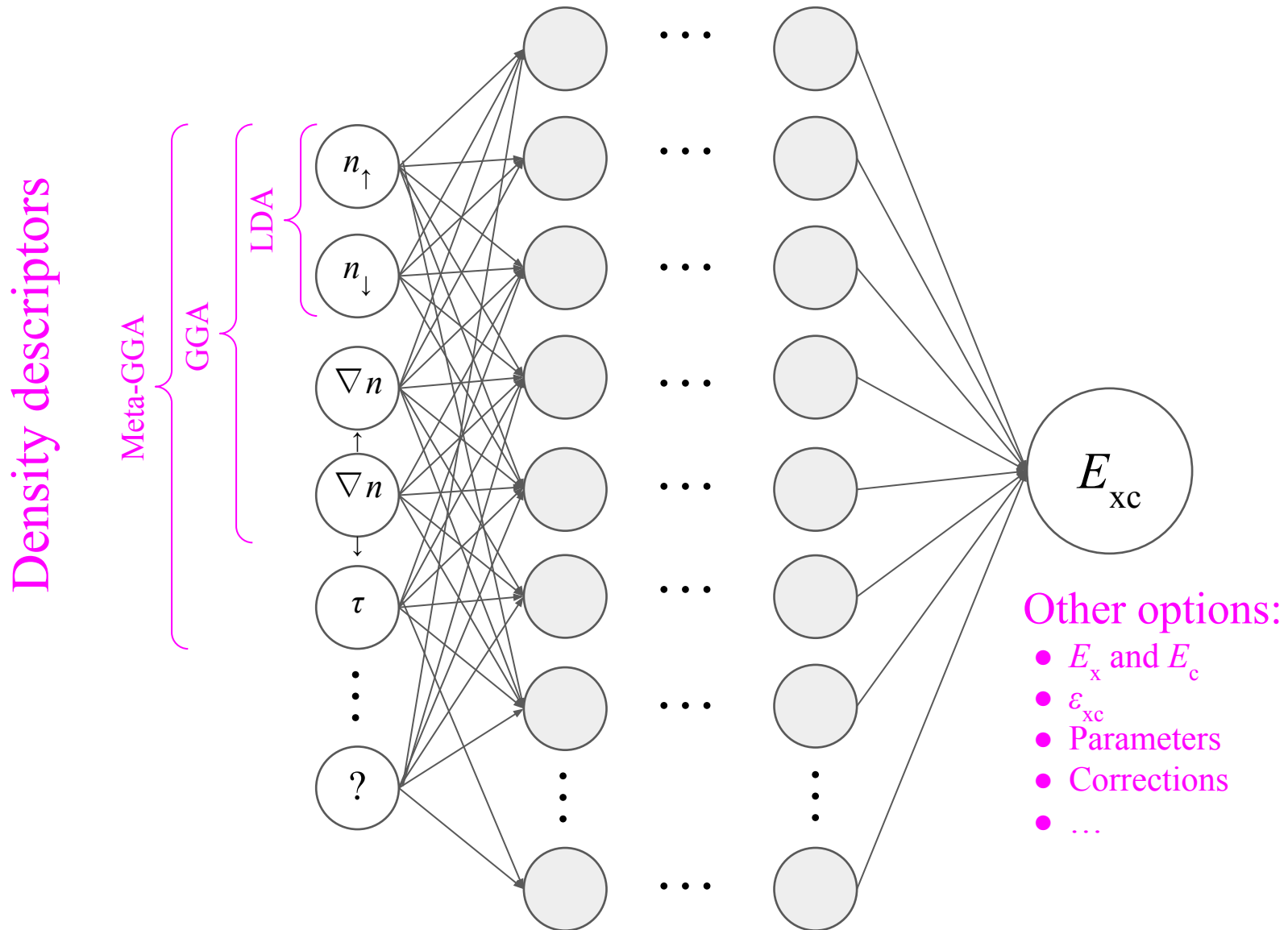
Feed Forward Neural Network



$$x'_i = f_{\text{activation}}(w_{ij}^1 x_j + b_i^1) \qquad f_i = f_{\text{activation}}(w_{ij}^3 x'_j + b_i^3)$$

$$x''_i = f_{\text{activation}}(w_{ij}^2 x'_j + b_i^2)$$

Artificial Neural Network as $E_{\mathbf{x}\mathbf{c}}$ Functionals



How do we get values for weights and biases?

You will need to define a loss function (Δ_{err}).

It should be:

- Be *quick to compute*,
- Represente the *physico-chemical accuracy* of the model

Δ_{err} defines an *order and equivalence relation* in the solution space, having a *global minimum* on the perfect model.

TRAINING = OPTIMIZING Δ_{err}

Relevant Features

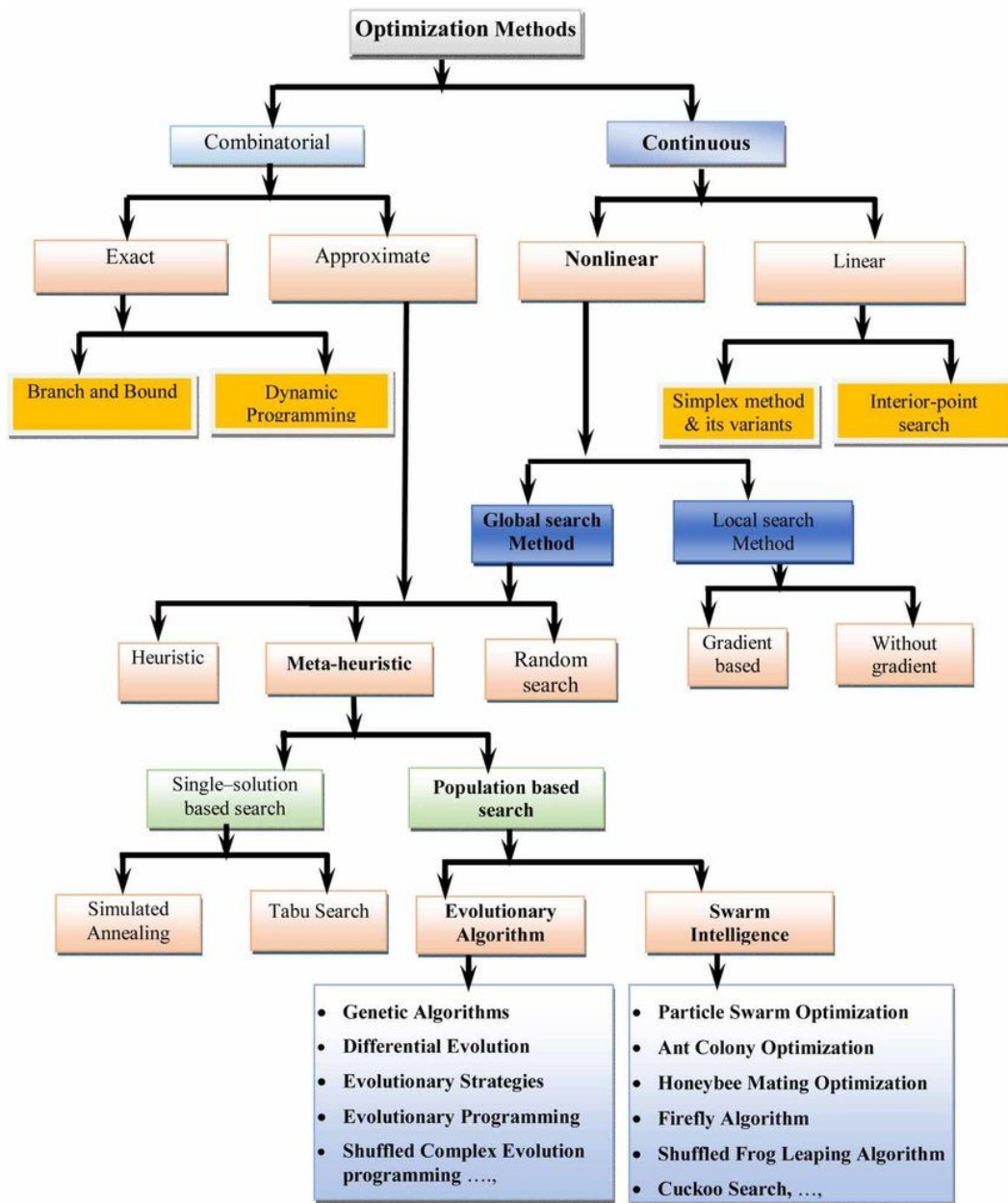
i.e., the quantities in which my training should be based on.

Shape of the Loss Function

i.e., the relations between the features and how they define a good functional.

Training Data Set

i.e., defining the data used on the calculation of the loss function.



$$\Delta_{\text{err}} = c_1 \sum_i |AE_i^{\text{ref}} - AE_i^{\text{DFT}}|$$

$$+ c_2 \sum_j \frac{1}{N_e} \int |\rho_j^{\text{ref}} - \rho_j^{\text{DFT}}| dV$$

Combinatorial or continuous?

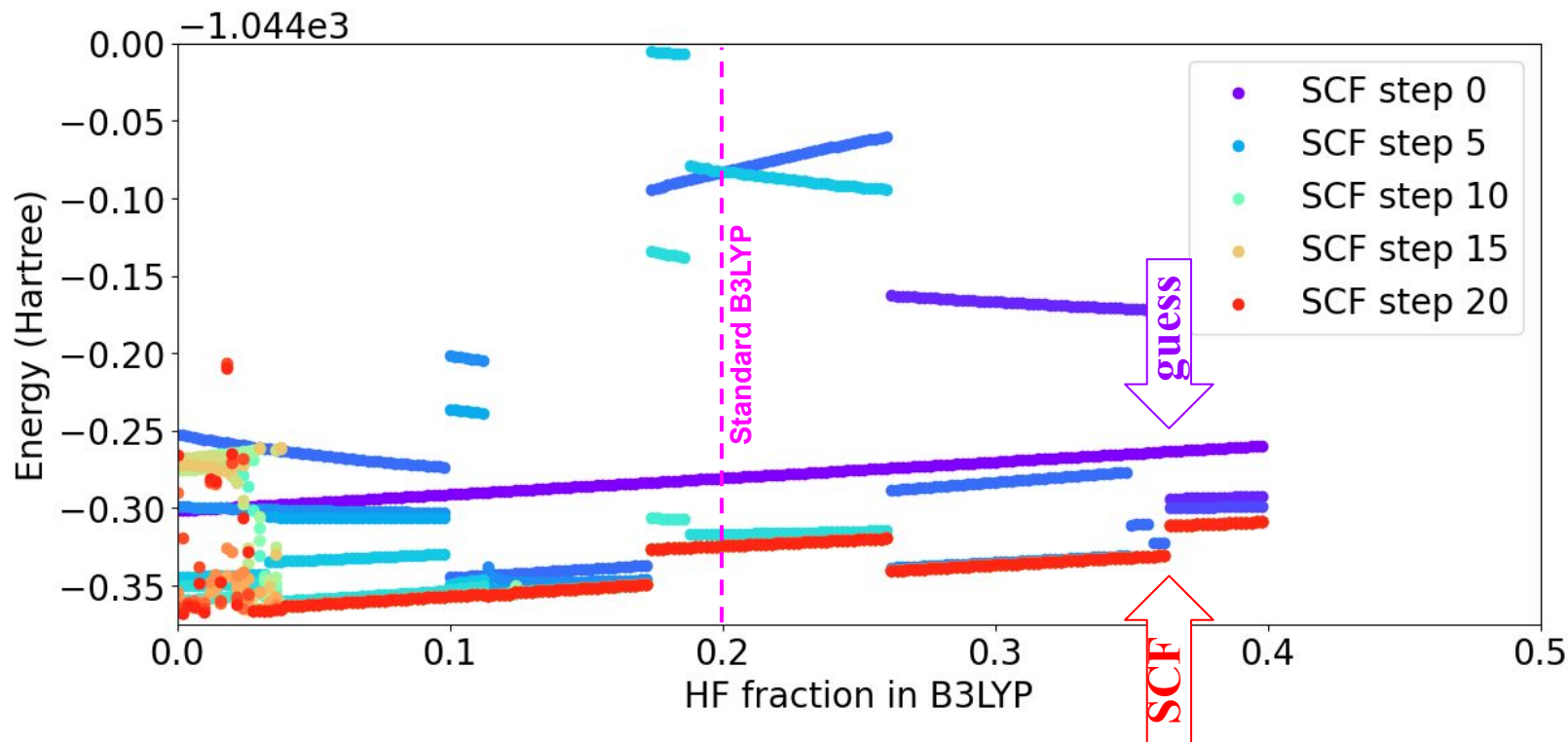
Non-gradient or gradient based?

Can we afford random search?

Problem computing gradients in the parameters of E_{xc}

The Loss function depends on the converged energy and density (ω -limit of the SCF) and those quantities are non-analytical in the parameters of the functional.

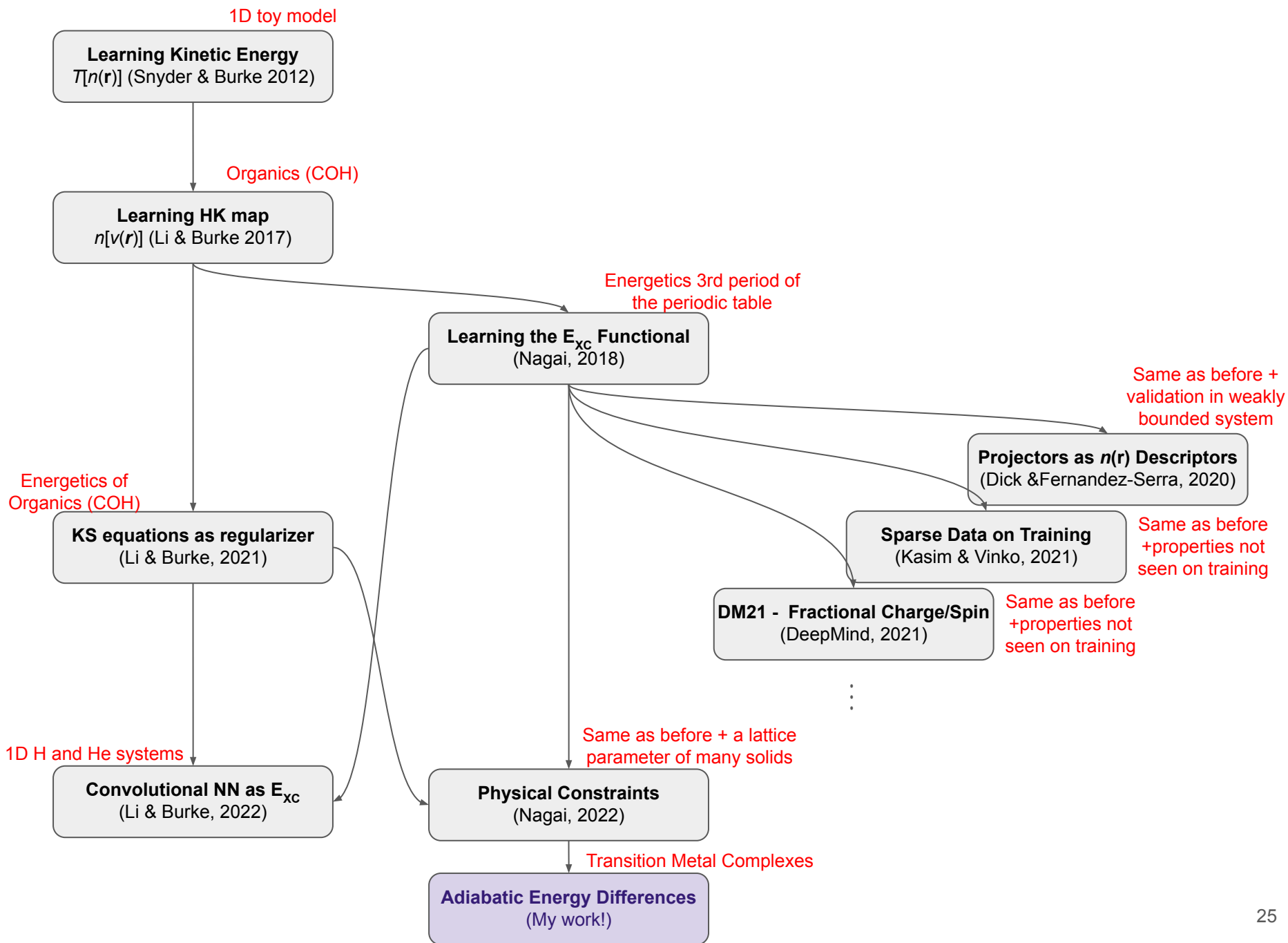
$$E_{xc}^{\text{B3LYP}} = \mathbf{x}E_x^{\text{HF}} + \mathbf{0.08}E_x^{\text{LDA}} + (1-\mathbf{0.08}-\mathbf{x})E_x^{\text{B88}} + \mathbf{0.81}E_c^{\text{LDA}} + (1-\mathbf{0.81})E_c^{\text{VWN}}$$



Cr^0

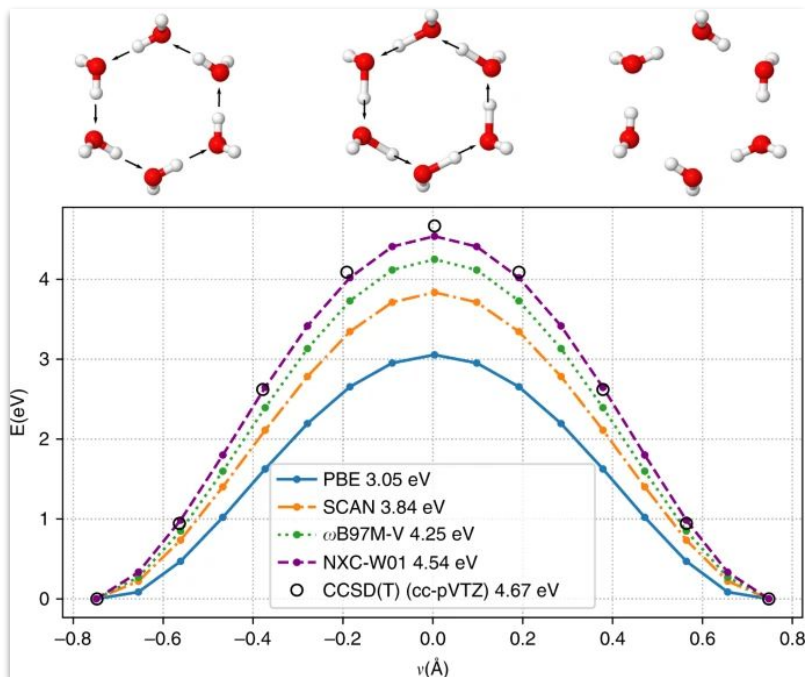
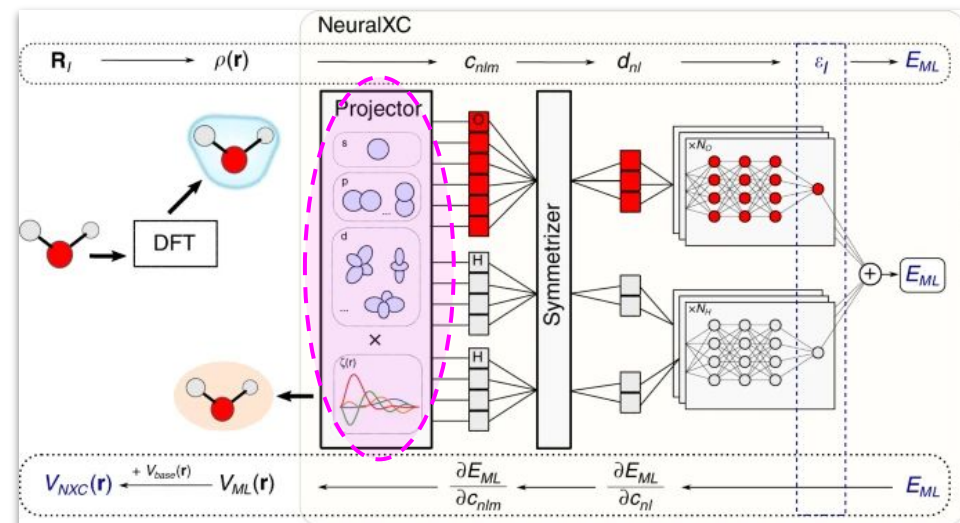
$2S+1=5$

‘huckel’
guess



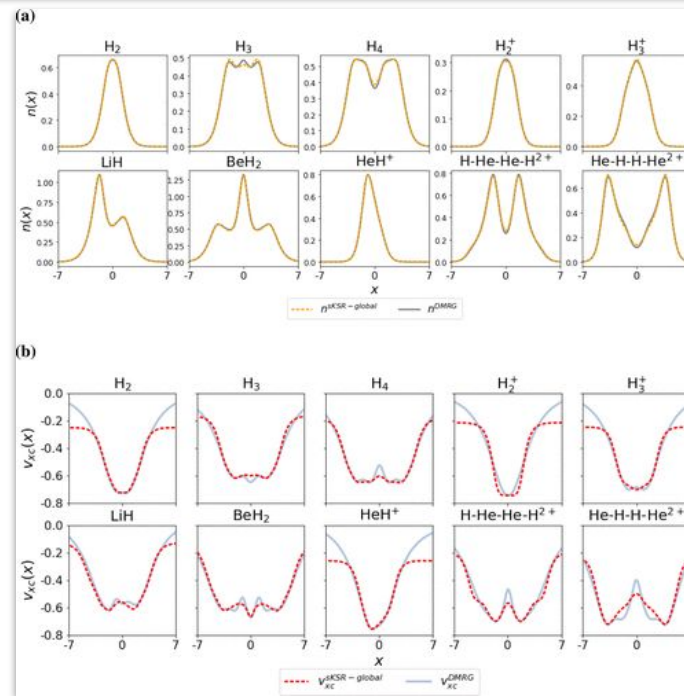
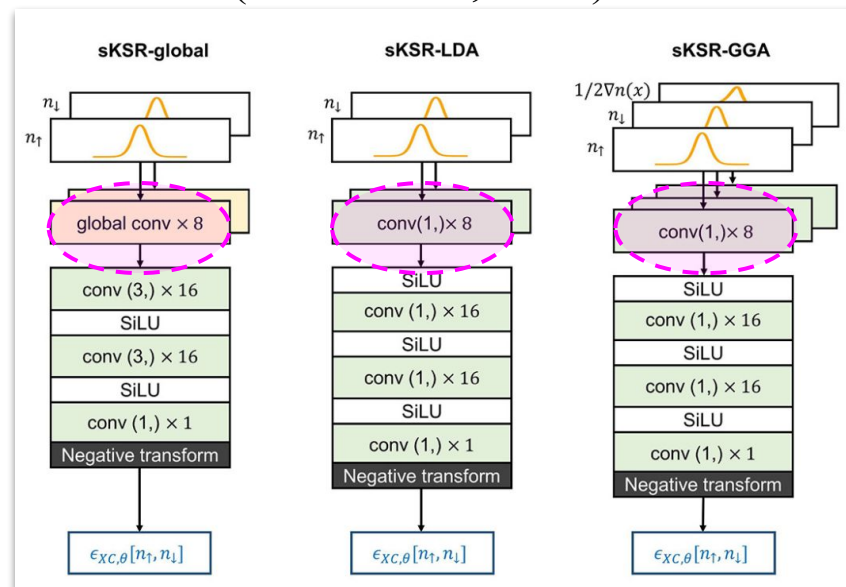
Projectors as n(r) Descriptors

(Dick & Fernandez-Serra, 2020)



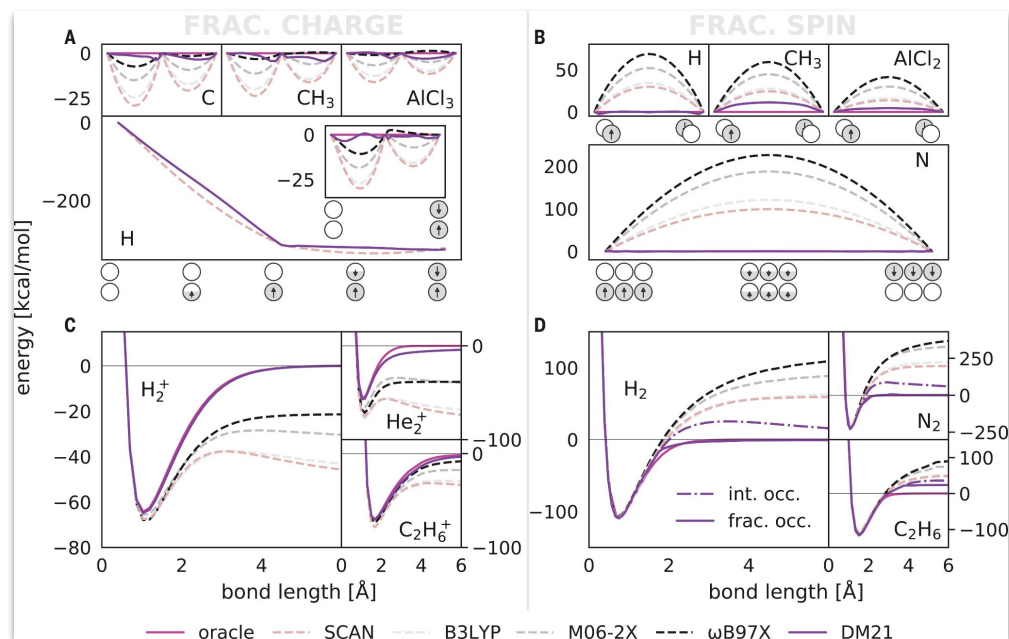
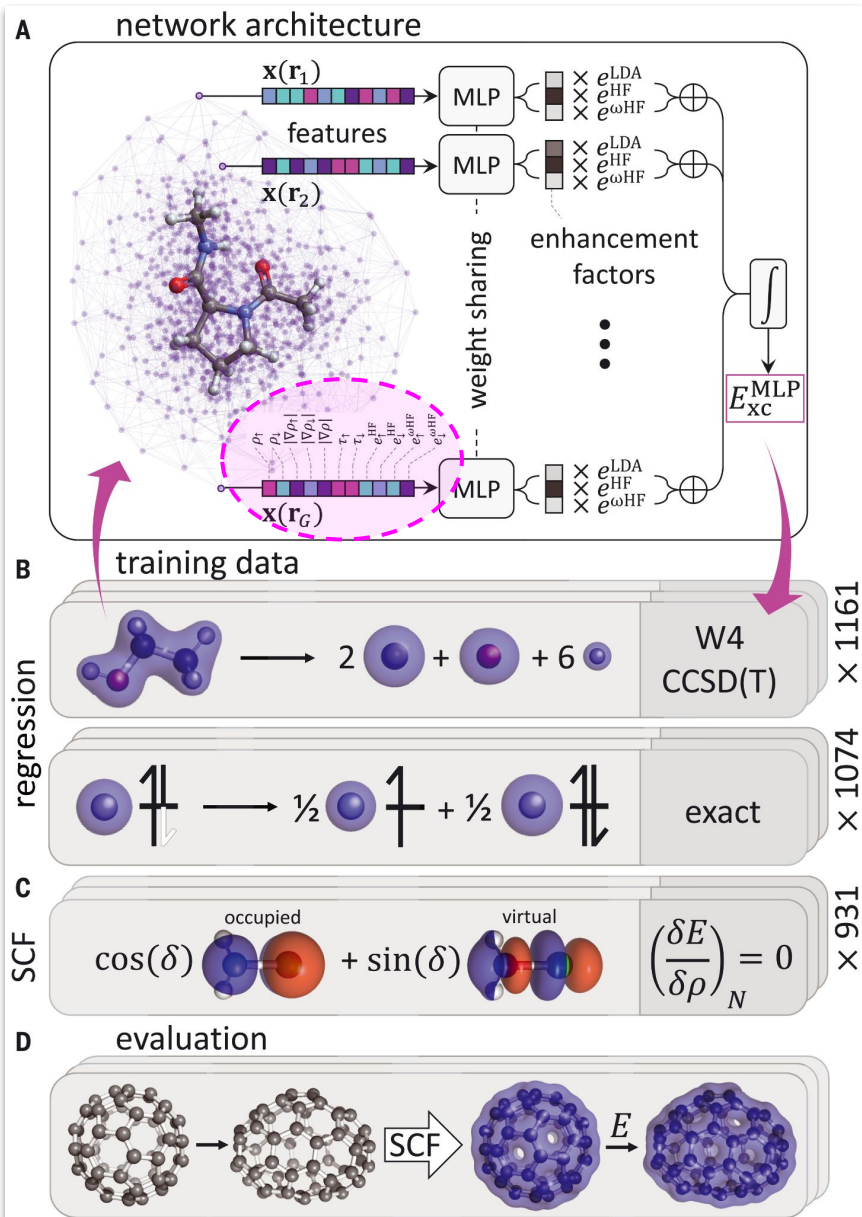
Convolutional NN as EXC

(Li & Burke, 2022)

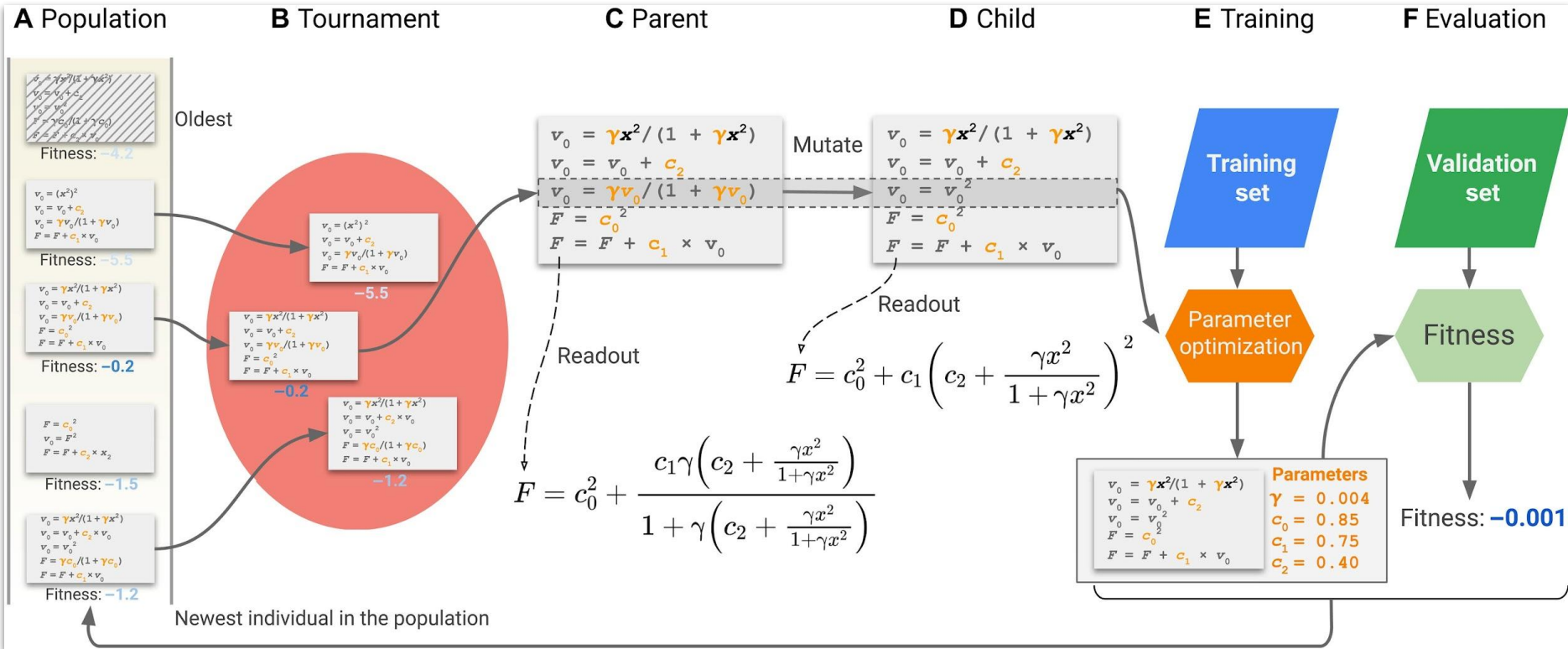


DM21 - DFT Functional from DeepMind, 2021

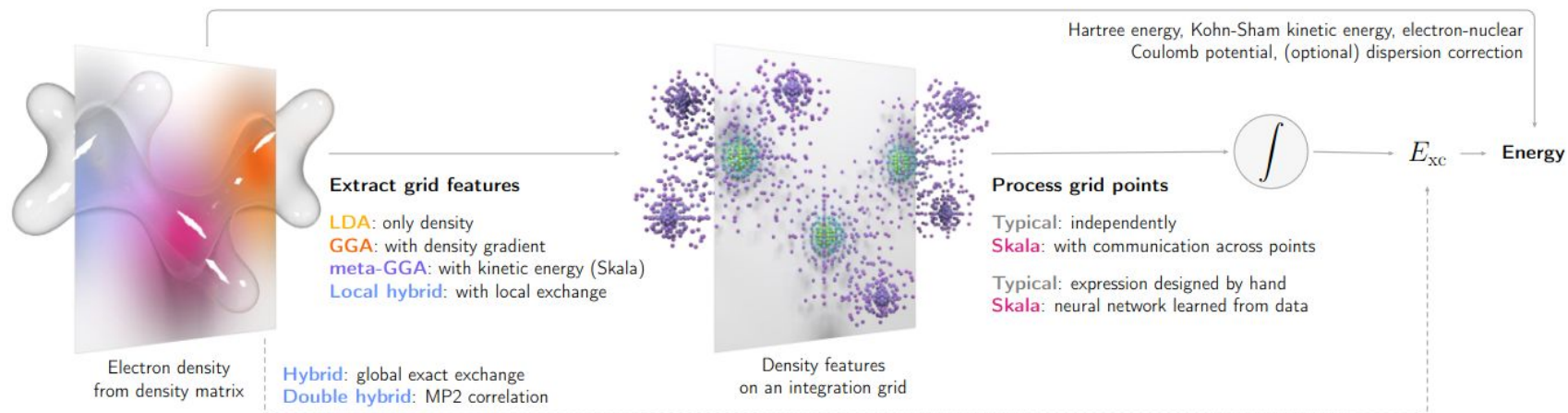
“The resulting functional, DM21 (DeepMind 21), correctly describes typical examples of **artificial charge delocalization** and **strong correlation** and performs better than traditional functionals on thorough **benchmarks** for **main-group atoms and molecules**.”



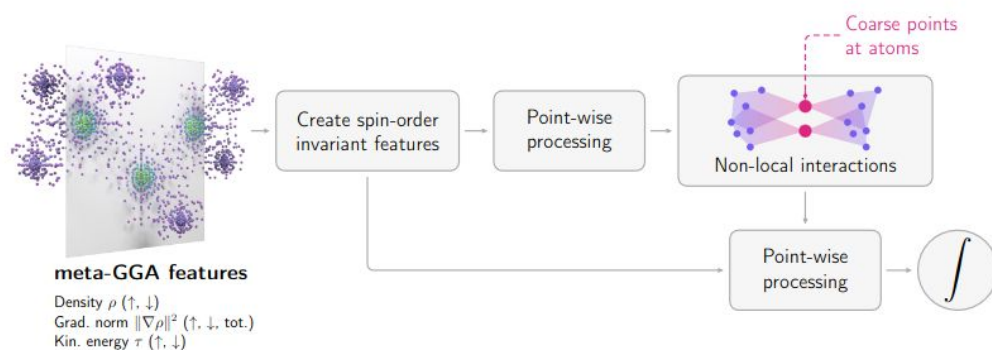
Google Accelerated Science 22



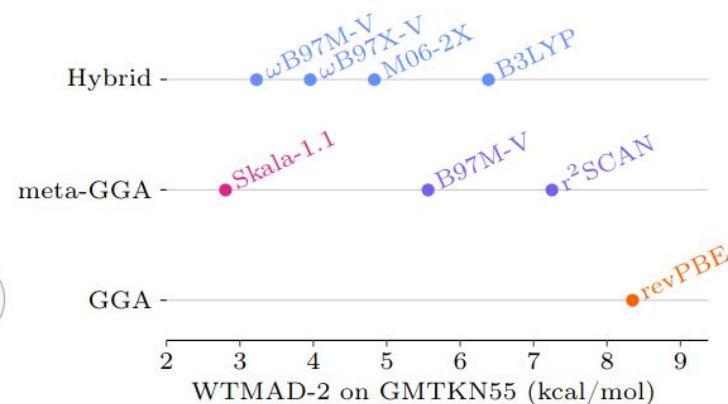
Skala Functional from Microsoft



(a) Anatomy of an exchange-correlation functional

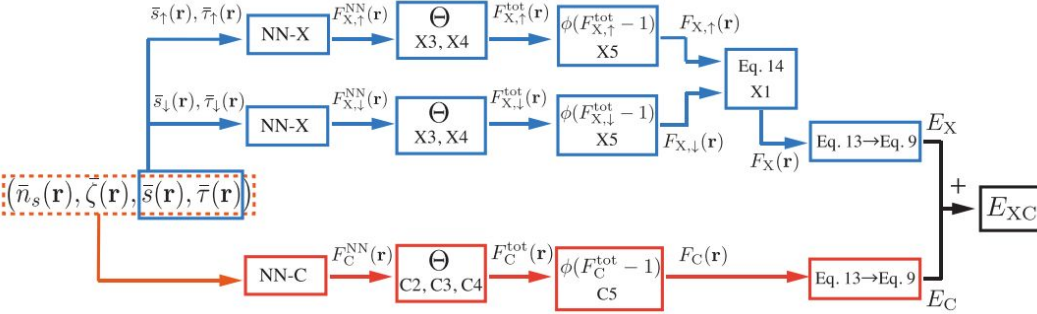


(b) **Skala**'s learned grid point processing



(c) Benchmark: errors for main-group chemistry

Physical Constraints (Nagai, 2022)



	Physical constraint	$\mathbf{x}_0$
X1	Correct uniform coordinate density-scaling behavior	
X2	Exact spin scaling relation	
X3	Uniform electron gas limit	$(0, 0)$
X4	F_X vanishes as $s^{-1/2}$ at $s \rightarrow \infty$	$(1, \bar{\tau})$
X5	Negativity of ε_X	
C1	Uniform electron gas limit	$(\bar{n}_s, \bar{\zeta}, 0, 0)$
C2	Uniform density scaling to low-density limit	$(0, \bar{\zeta}, \bar{s}, \bar{\tau})$
C3	Weak dependence on ζ in low-density region	
C4	Uniform density scaling to high-density limit	$(1, \bar{\zeta}, \bar{s}, \bar{\tau})$
C5	Nonpositivity of ε_C	

TABLE II. Benchmark results for atomization energies of 144 molecules. (MAE, mean absolute error; ME, mean error; SD, standard deviation of signed error.)

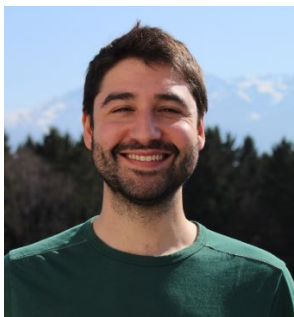
XC	MAE (kcal/mol)	ME (kcal/mol)	SD (kcal/mol)
PBE	17.3	16.2	13.1
SCAN	6.2	-4.5	5.7
NN-based	4.8	1.8	6.3
pcNN-based	3.6	0.3	4.5

TABLE III. Benchmark results for lattice constants of 48 solids. Parentheses in the “NN-based” row indicate that the numerical calculations for six solids did not converge; thus, only the converged calculations were used for the statistics.

XC	MAE (mÅ)	ME (mÅ)	SD (mÅ)
PBE	38.1	33.9	44.2
SCAN	22.3	-7.5	28.5
NN-based	(22.9)	(0.8)	(32.0)
pcNN-based	19.1	-2.5	26.5

Artificial Neural Network-Based Density Functional Approach for Adiabatic Energy Differences in Transition Metal Complexes

Others involved
in this work:



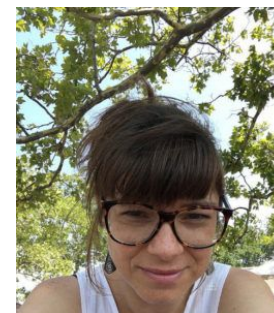
Lorenzo Mariano
(Postdoc, Trinity
College)



Emilie Devijver
(CNRS, LIG)



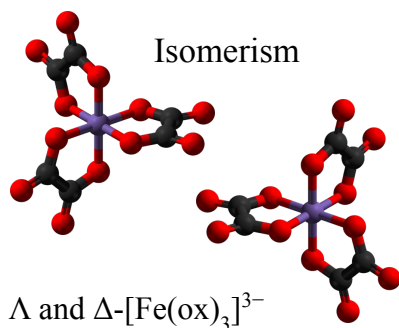
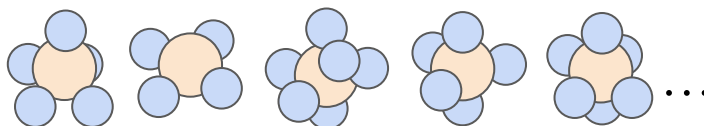
Noel Jakse
(G-INP, SIMaP)



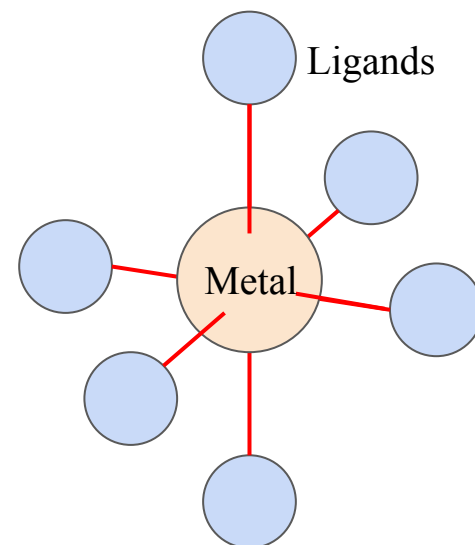
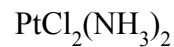
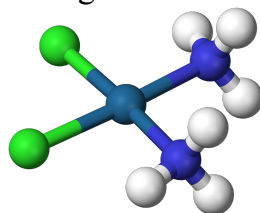
Roberta Poloni
(CNRS, SIMaP)

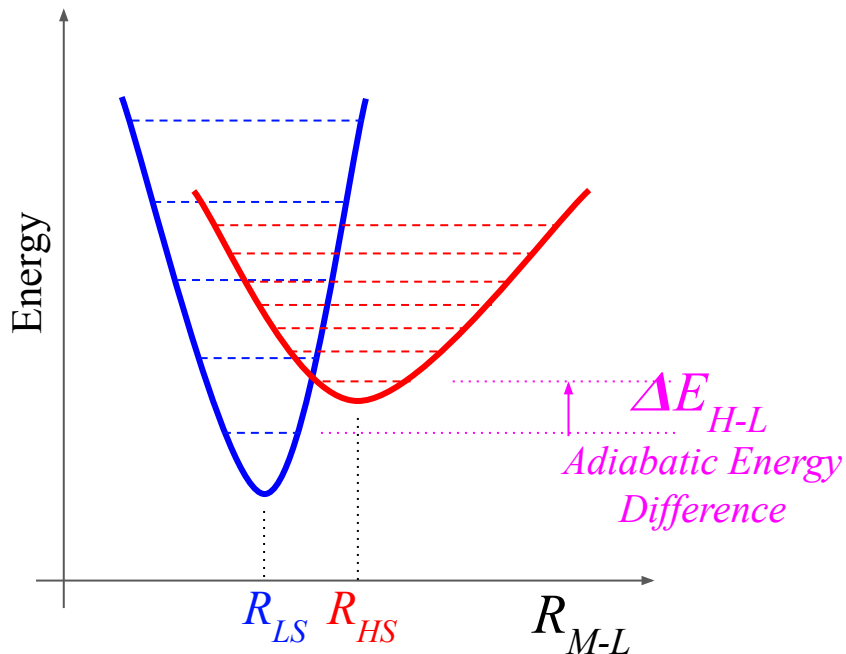
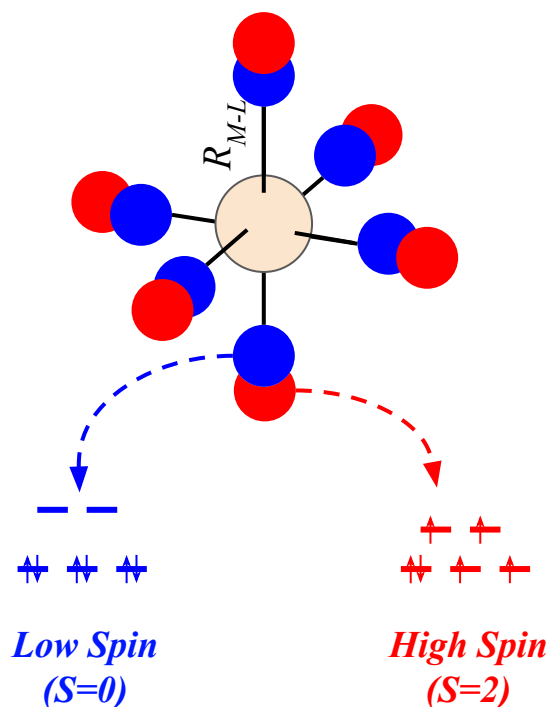
J. Chem. Theory Comput. **19**, 21 (2023): 7555-7566.
Available at [arXiv:2304.07899] or [DOI:10.1021/acs.jctc.3c00600]

Many possible symmetries:



Different ligands combination



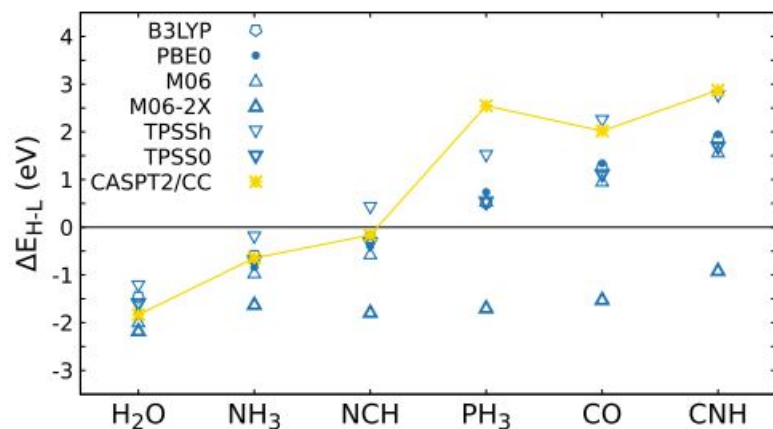
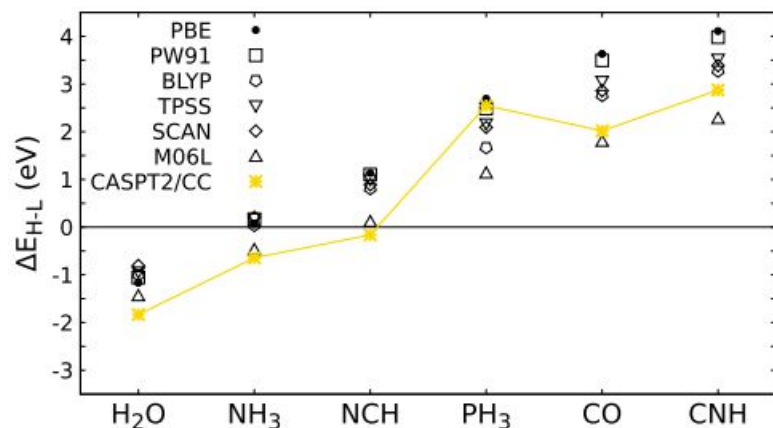
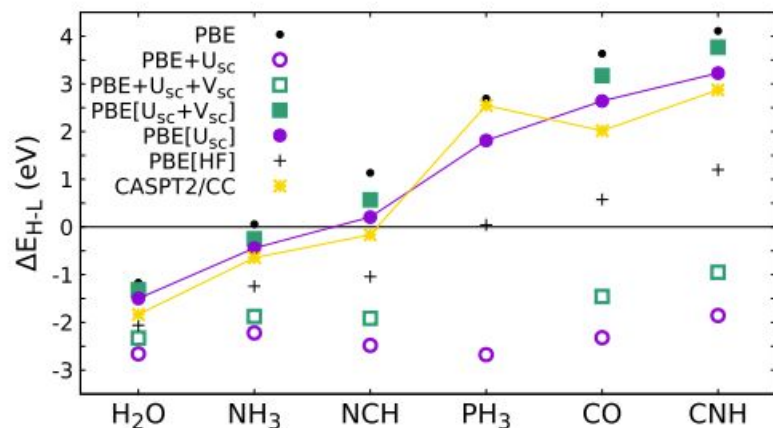


Spin crossover (SCO) is a phenomenon that occurs in some metal complexes where the spin state of the complex changes due to an external stimulus

Adiabatic energy difference: energy difference between two spin states computed at the corresponding geometry

$$\Delta E_{H-L} = E_{HS} - E_{LS}$$

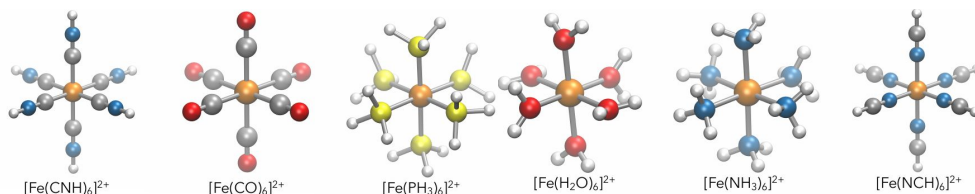
Challenge for any ab initio methods → **Lack of error cancellation**



CASPT2/CC is a multiconfigurational method that we take as reference for ΔE_{H-L} (expensive)

Different density functionals yield very different values of ΔE_{H-L} .

Can a DFT functional be build (using machine learning) to predict accurate values of ΔE_{H-L} ?



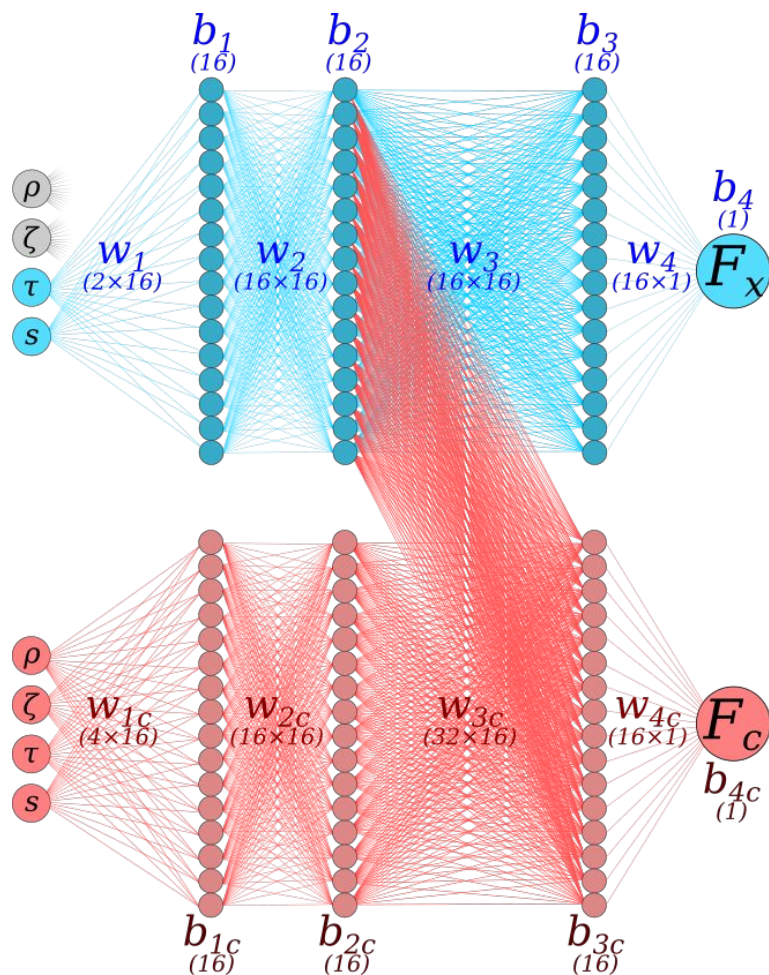
L. A. Mariano et al. *Journal of Chemical Theory and Computation* 17.5 (2021): 2807-2816.

Our Artificial Neural Network Functional

Architecture

$$E_{XC}[\rho] = \int \rho(\mathbf{r}) \epsilon_{XC}(\mathbf{g}[\rho(\mathbf{r})]) d\mathbf{r}$$

$$\epsilon_{XC} = F_X \epsilon_X^{\text{R2SCAN}} + F_C \epsilon_C^{\text{R2SCAN}}$$



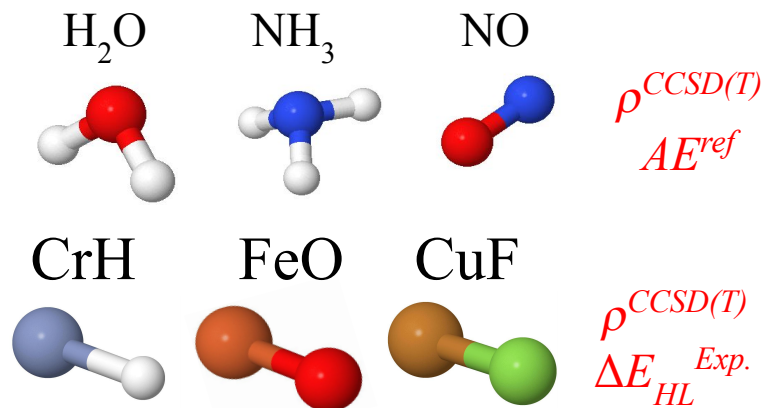
1503 parameters

Loss Function

$$\begin{aligned} \Delta_{\text{err}} = & c_1 \sum_i |AE_i^{\text{ref}} - AE_i^{\text{DFT}}| \\ & \text{Atomization Energies (AE)} \\ & + c_2 \sum_j \frac{1}{N_e} \int |\rho_j^{\text{ref}} - \rho_j^{\text{DFT}}| dV \\ & \text{Electronic Densities } (\rho) \\ & + c_3 \sum_k |\Delta E_{HL,k}^{\text{ref}} - \Delta E_{HL,k}^{\text{DFT}}| \\ & \text{Adiabatic Energy Differences } (\Delta E_{HL}) \end{aligned}$$

Training set

Different combinations of:

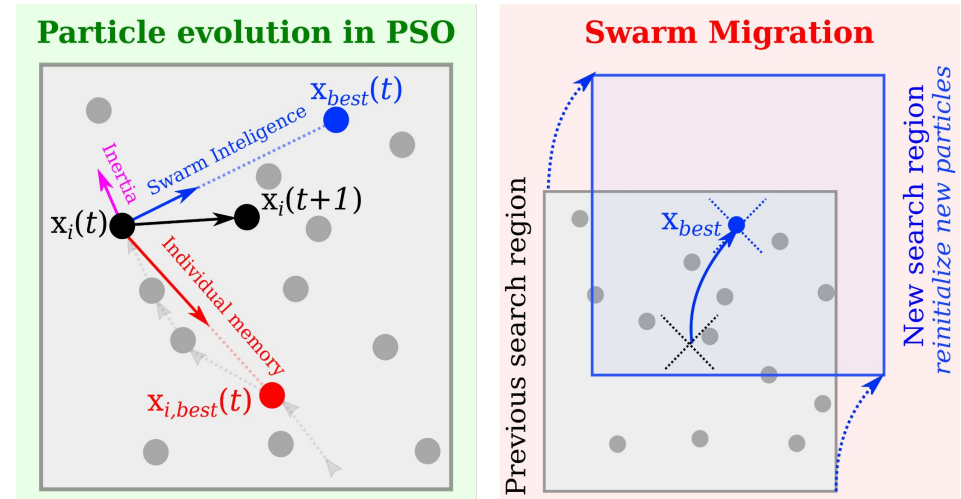


Training Algorithm: Modified PSO via Migrations

Idea:



In practice:



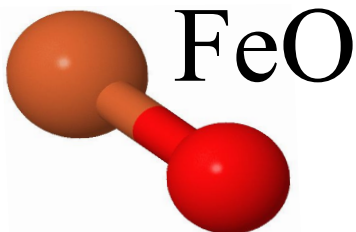
Particle Swarm Optimization (PSO)

- No need of evaluating the derivatives of the loss function with respect to the parameters of the functional
- Improves over Monte Carlo optimization exploring the phenomena of swarm intelligence

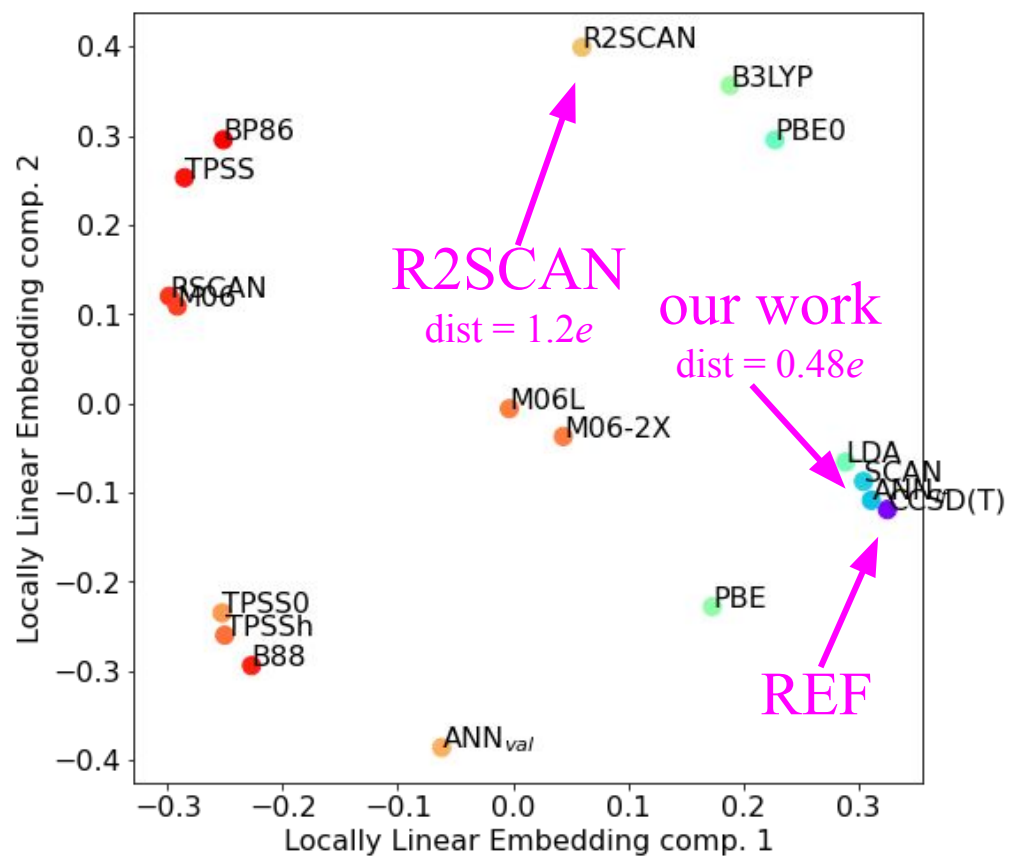
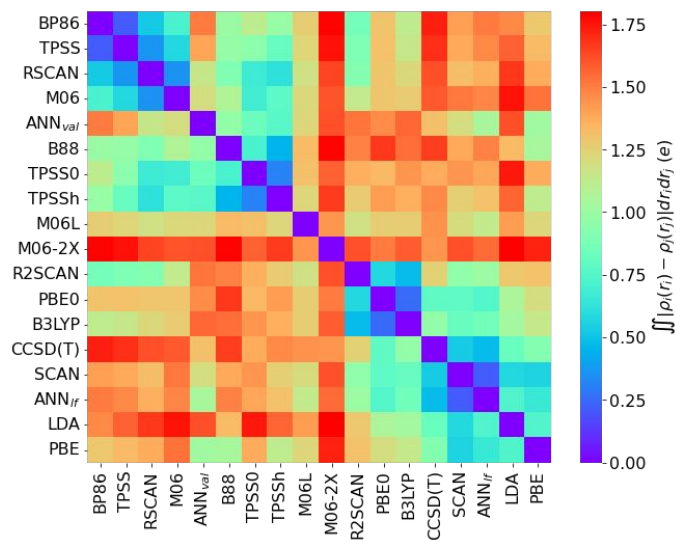
Migrations

- Changing periodically the search box to follow the best solution
- Improves on the stability not generating searches too far from the best solution found in the present epoch

The trained functional predicts densities closer to the CCSD(T) reference than to the original R2SCAN.



$$\text{dist}_{ij} = \iint |\rho_i(\mathbf{r}) - \rho_j(\mathbf{r})| d\mathbf{r}$$



Validation Set: Iron +2 Complexes

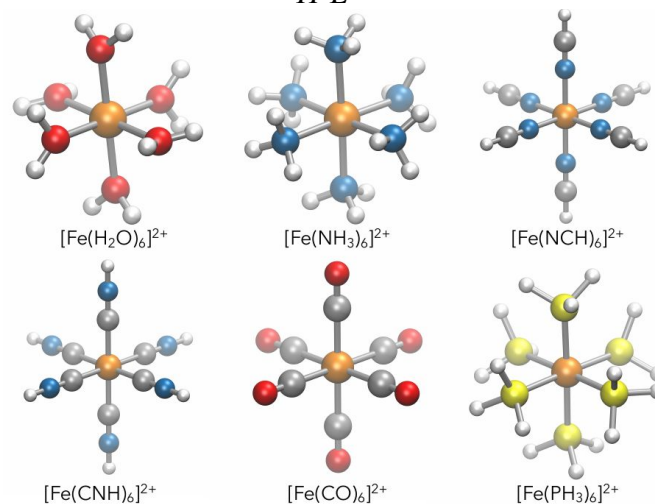
	ΔE_{H-L} (eV)									
	PBE	SCAN	NAS ^a	B3LYP	DM21	R2SCAN	PBE[U] ^b	AN^N_{llf}	AN^N_{val}	REF. ^c
$[\text{Fe}(\text{H}_2\text{O})_6]^{2+}$	-1.17	-0.81	-0.74	-1.44	-0.63	-1.36	-1.50	-2.51	-1.83	-1.83
$[\text{Fe}(\text{NH}_3)_6]^{2+}$	-0.06	0.21	0.21	-0.58	0.26	-0.29	-0.44	-1.27	-0.71	-0.64
$[\text{Fe}(\text{NCH})_6]^{2+}$	1.14	0.89	0.90	-0.21	0.64	0.41	0.21	-0.39	0.08	-0.16
$[\text{Fe}(\text{PH}_3)_6]^{2+}$	2.69	2.09	2.21	0.62	2.24	1.91	1.81	2.05	1.94	2.54
$[\text{Fe}(\text{CO})_6]^{2+}$	3.63	2.86	2.93	1.25	2.35	2.58	2.64	2.42	2.50	2.02
$[\text{Fe}(\text{CNH})_6]^{2+}$	4.11	3.38	3.42	1.86	2.99	3.06	3.23	2.93	2.99	2.87
MAE^c	0.92	0.79	0.80	0.70	0.61	0.46	0.44	0.41	0.25	

^a NAS is the functional obtained by R. Nagai, R. Akashi, and O. Sugino.²⁹

^b PBE[U] represents the use of Hubbard U_{sc} -corrected density in the PBE functional.²⁸

^c Reference values come from CASPT2/CC.²⁸

Ref. for ΔE_{H-L} : CASPT2/CC



29 - Nagai, R., Akashi, R., and Sugino, O. "Machine-learning-based exchange correlation functional with physical asymptotic constraints." *Physical Review Research*, 4.1 (2022): 013106.

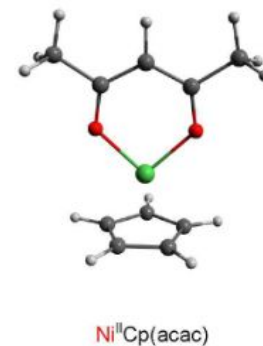
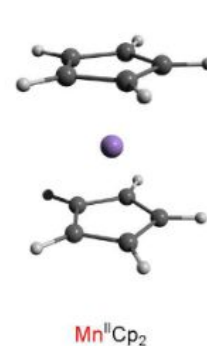
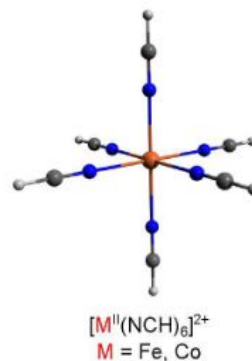
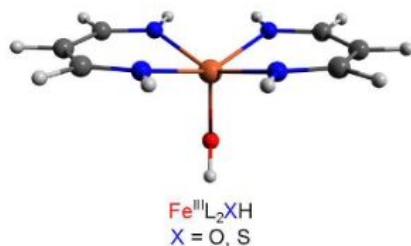
28 - Mariano, Lorenzo A., Bess Vlasisavljevich, and Roberta Poloni. "Improved Spin-State Energy Differences of Fe (II) molecular and crystalline complexes via the Hubbard U-corrected Density." *Journal of Chemical Theory and Computation* 17.5 (2021): 2807-2816.

Other Metal Complexes

		ΔE_{H-L} (eV)							
	2S+1	ANN_{lf}^{ALL}	ANN_{val}^{ALL}	R2SCAN	TPSS	PBE	PBE[U]	DM21	REF. ^b
FeL_2	1 $\rightarrow$ 5	-1.439	-0.944	-0.611	0.227	-	-1.096	-1.968	-1.487
	3 $\rightarrow$ 5	0.193	0.513	0.746	1.259	1.245	0.756	0.814	0.213
MnL_2	2 $\rightarrow$ 6	-2.078	-1.427	-0.957	-0.055	-0.111	-0.961	-	-1.782
	4 $\rightarrow$ 6	-0.061	0.207	0.414	0.835	0.919	0.491	-0.739	-0.455
FeL_2SH	2 $\rightarrow$ 6	-0.464	0.081	0.470	-0.416	1.087	-	-	0.399
	4 $\rightarrow$ 6	-0.488	-0.130	0.123	0.588	0.521	0.245	-	-0.017
$[Co(NCH)_6]^{2+}$	2 $\rightarrow$ 4	-0.372	-0.170	-0.027	0.31	0.476	0.165	0.057	-0.581
$NiCp(acac)$	1 $\rightarrow$ 3	-0.204	-0.177	-0.157	0.046	0.163	0.095	0.414	0.117
$MnCp_2$	2 $\rightarrow$ 6	-0.220	-0.354	0.184	0.649	0.981	0.432	-	0.304
MAE^b		0.349	0.406	0.474	0.945	0.885 ^a	0.482 ^a	0.460 ^a	

^a Average computed over available values.

^b Reference values come from CASPT2/CC. ⁶⁸



Molecules from: "Spin state energetics in first-row transition metal complexes: contribution of (3s3p) correlation and its description by second-order perturbation theory."

Kristine Pierloot, Quan Manh Phung, and Alex Domingo

Journal of chemical theory and computation 13.2 (2017): 537-553.

Preparing for the Practical Section

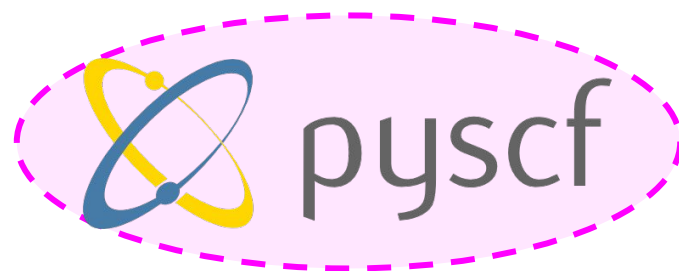
“Here be Dragons”

Quantum Chemistry Software for DFT

- **Abinit** - a software suite to calculate the optical, mechanical, vibrational, and other observable properties of materials
- **ACE-Molecule** - a quantum chemistry package based on a real-space numerical grid
- **ADF** - a density functional theory program for molecules and condensed matter
- **APE** - a computer package designed to generate and test norm-conserving pseudopotentials within density functional theory
- **AtomPAW** - a program for generating projector augmented wave functions
- **BAGEL** - a parallel electronic-structure program
- **BigDFT** - a fast, precise, and flexible density functional theory code for ab-initio atomistic simulation
- **CP2K** - a program to perform atomistic and molecular simulations of solid state, liquid, molecular, and biological systems
- **DFT-FE** - a massively parallel real-space code for first principles based materials modelling using Kohn-Sham density functional theory
- **DP** - a linear response time-dependent density functional theory code with a plane wave basis set
- **Chronus Quantum** - a computational chemistry software package focused on explicitly time-dependent and post-SCF methods
- **Elk** - an all-electron full-potential linearised augmented-plane wave code
- **entos** - a software package for Gaussian-basis ab initio molecular dynamics calculations on molecular and condensed-phase chemical reactions and other processes
- **ERKALE** - a DFT/HF molecular electronic structure code based on Gaussian-type orbitals
- **exciting** - a full-potential all-electron density-functional-theory package implementing the families of linearized augmented planewave methods
- **FHI-AIMS** - an efficient, accurate, all-electron, full-potential electronic structure code package for computational molecular and materials science
- **GAMESS (US)** - a general ab initio quantum chemistry package
- **GPAW** - a density-functional theory Python code based on the projector-augmented wave method
- **HelFEM** - Finite element methods for electronic structure calculations on small systems
- **Horton** - Python development platform for electronic structure methods
- **INQ** - a modern GPU-accelerated computational framework for (time-dependent) density functional theory
- **JDFTx** - plane-wave code designed for joint density functional theory
- **MADNESS** - a multiwave adaptive numerical grid program for electron
- **MOLGW** - many-body perturbation theory for atoms, molecules, and clusters
- **Molpro** - a comprehensive system of ab initio programs for advanced molecular electronic structure calculations
- **MRCC** - a suite of ab initio and density functional quantum chemistry programs for high-accuracy electronic structure calculations
- **NWChem** - an open source, high-performance computational chemistry program
- **Octopus** - a program aimed at the ab initio virtual experimentation on a hopefully ever-increasing range of system types
- **OpenMolcas** - a quantum chemistry software package specializing in multiconfigurational approaches
- **ORCA** - ab initio quantum chemistry program that contains modern electronic structure methods
- **PROFESS** - orbital-free density functional theory implementation to simulate condensed matter and molecules
- **Psi4** - an open-source suite of ab initio quantum chemistry programs designed for efficient, high-accuracy simulations of molecular properties
- **PySCF** - Python-based Simulations of Chemistry Framework
- **QuantumATK** - code including pseudopotential-based density functional theory methods with LCAO and plane-wave basis sets in one framework
- **Quantum Espresso** - an integrated suite of open source computer codes for electronic-structure calculations and materials modeling at the nanoscale
- **Turbomole** - a program package for electronic structure calculations
- **Vasp** - the Vienna Ab initio Simulation Package: atomic scale materials modelling from first principles
- **WIEN2k** - program for electronic structure calculations of solids using density functional theory based on the full-potential (linearized) augmented plane-wave + local orbitals method
- **Yambo** - a program that implements many-body perturbation theory methods such as GW and BSE and time-dependent density functional theory

LIBXC

<https://tddft.org/programs/libxc/>



WE
GAMESS

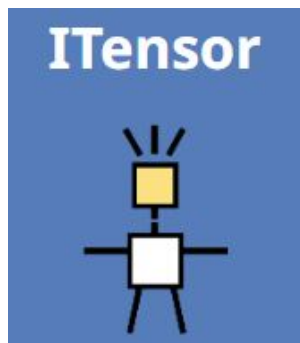


CP2K



Some tools and libraries for Neural Networks

 PyTorch  TensorFlow



theano

Caffe  Keras



What we will see on the Practical Section:

1. Basics of PySCF

- Running basic calculations using DFT to compute energies of molecular systems
- Additional informations on setting tricky calculations for charged/magnetic systems and controlling output options.

2. Running PySCF DFT with a customized functional

- Exemple where we code PW86 functional by hand and mix it to PBE, highlighting the details on how it is done.

3. Crafting and training a basic multilayer perceptron with TensorFlow

- Sample code of a very elementar implementation of a feed forward neural network using Tensorflow

4. Using a multilayer perceptron as functional

- Writing a external functional in PySCF that uses a Neural Network to correct PBE.

5. Using a pre-trained ANN functional

- We will use PySCF to run some computations with a Neural Network correction PBE trained only using H₂O atomization energy.

The main goal of this tutorial is to allow you to comprehend how to implement and use a neural network functional.

This tutorial uses the very simple case of an machine learning functional that correct PBE and try to predict energies closer to the B3LYP ones for molecules only composed of O and H atoms.

Even with its simplicity, this exemple show all the tools and elements necessary to extend to more complex cases.

What you will need:

I strongly recommend to use the Google Colab!

You will need internet and maybe a google account...

pyscf <https://pyscf.org/install.html>

tensorflow <https://www.tensorflow.org/>

If you want to do the things in the harder way:

<https://pyscf.org/install.html#pyscf-docker-image>

Thanks for your attention!

DON'T
FORGET TO
LOVE EACH
OTHER



<https://github.com/jpalastus>
joao-paulo.almeida-de-mendonca@grenoble-inp.fr

@alastus